

고성능 RAG를 위한 최신 테크닉

강의 개요

강사님 소개

곽기은 님

학부 : Economics, Business analytics

석사 : Master's in analytics

재보험사 MunichRe, Data scientist

SK Discovery, Data scientist

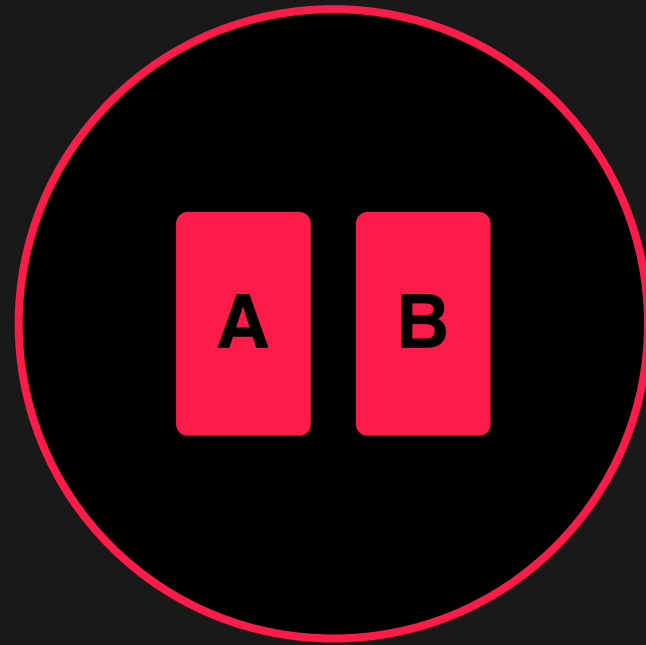


Chapter overview



Raptor RAG

특징 :
활용 방법 :



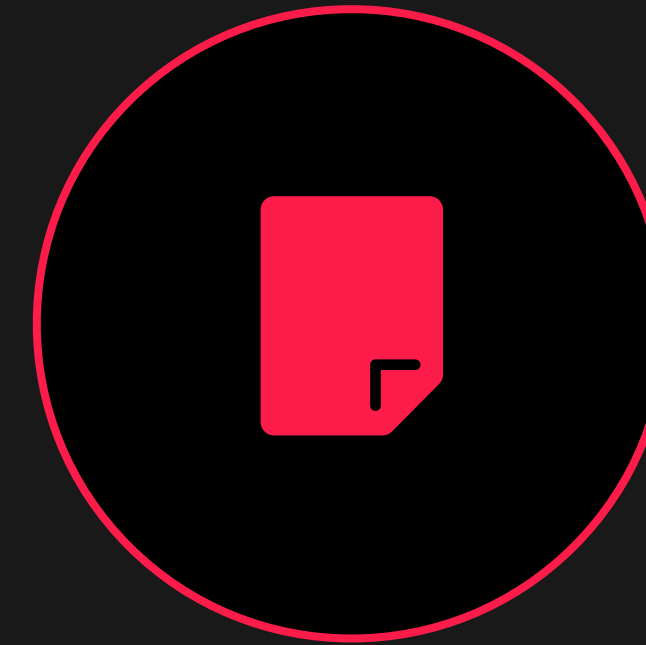
Self RAG

특징 :
활용 방법 :



Corrective RAG

특징 :
활용 방법 :



Adaptive RAG

특징 :
활용 방법 :



Multi hop RAG

특징 :
활용 방법 :



Tree RAG

특징 :
활용 방법 :

Lesson

RAG를 활용하여 '정보'를 '지식'으로 변환 할 수 있는 능력

RAG의 필요성

01

ROI

Cost Efficiency

- General-purpose LLM을 활용하여 컴퓨팅 리소스, 인력, 시간, 예산 등 자원을 효율적으로 활용 가능

모델 운영 관리 및 업데이트

- LLM 모델을 retraining 하기보다는 knowledge base 활용 파이프라인 자체를 운영 관리 하여 최신성 유지 가능

02

Scalability

Agile 방식의 개발

- General-purpose LLM을 활용하여 LLM service를 빠르게 구축 할 수 있음

Flexible integration

- Retrieval database를 업데이트 하여 다양한 도메인에 특화된 서비스 개발 가능

03

Performance

Enhanced Accuracy

- 필요한 정보를 유동적으로 retrieve 하여 bias나 hallucination이 포함된 아웃풋 생성을 방지할 수 있음

Handling Specialized Information

- General-purpose LLM에 없는 특수한 정보를 활용하는 서비스 개발 가능

Modular RAG

RAG의 시작

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

John Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Hein

Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe

facebook AI Research; [†]University College London; ^{*}New York University
plewis@fb.com

Abstract

...

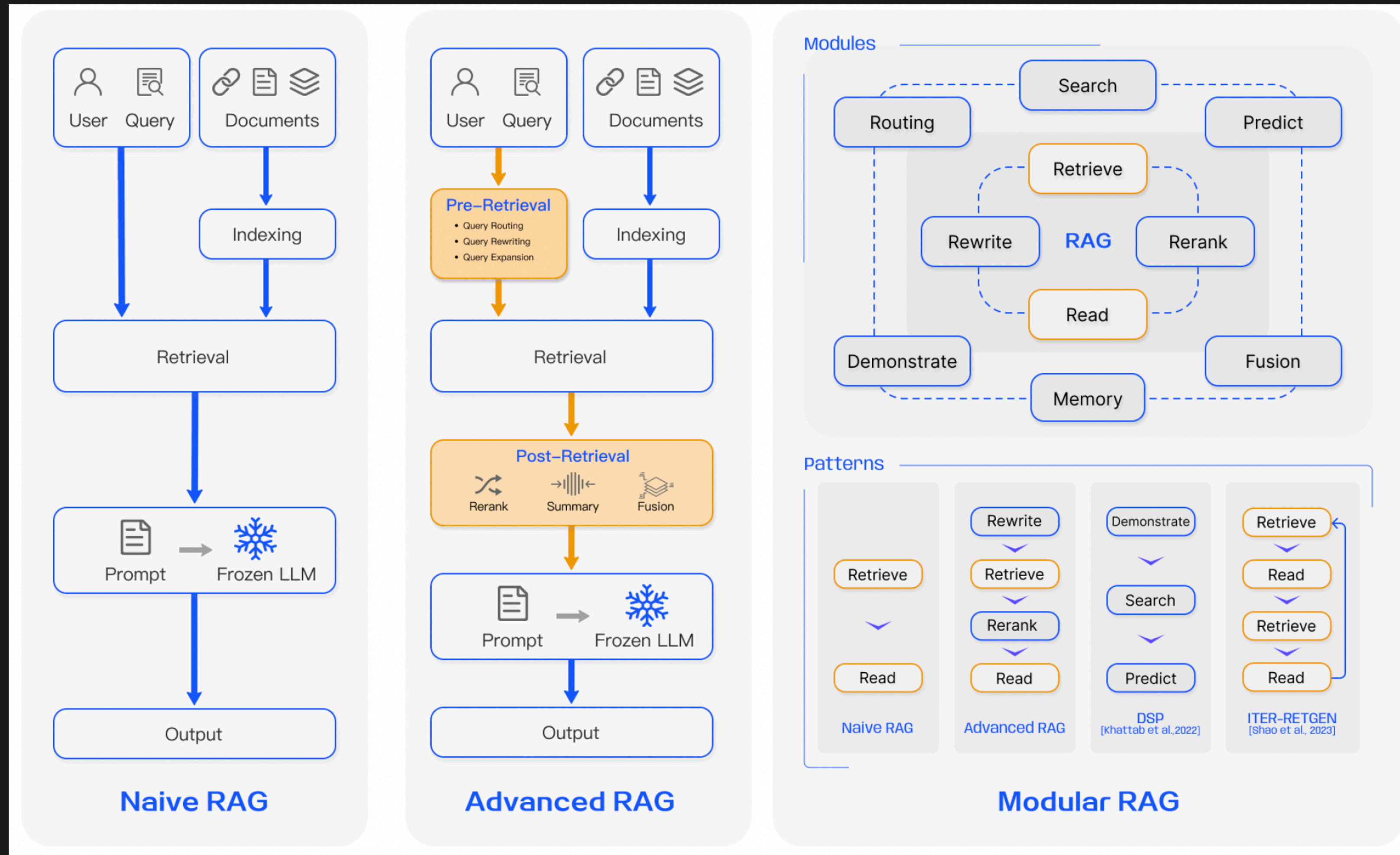
We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever.

...

For language generation tasks, we find that RAG models generate more specific, diverse and factual language than a state-of-the-art parametric-only seq2seq baseline.

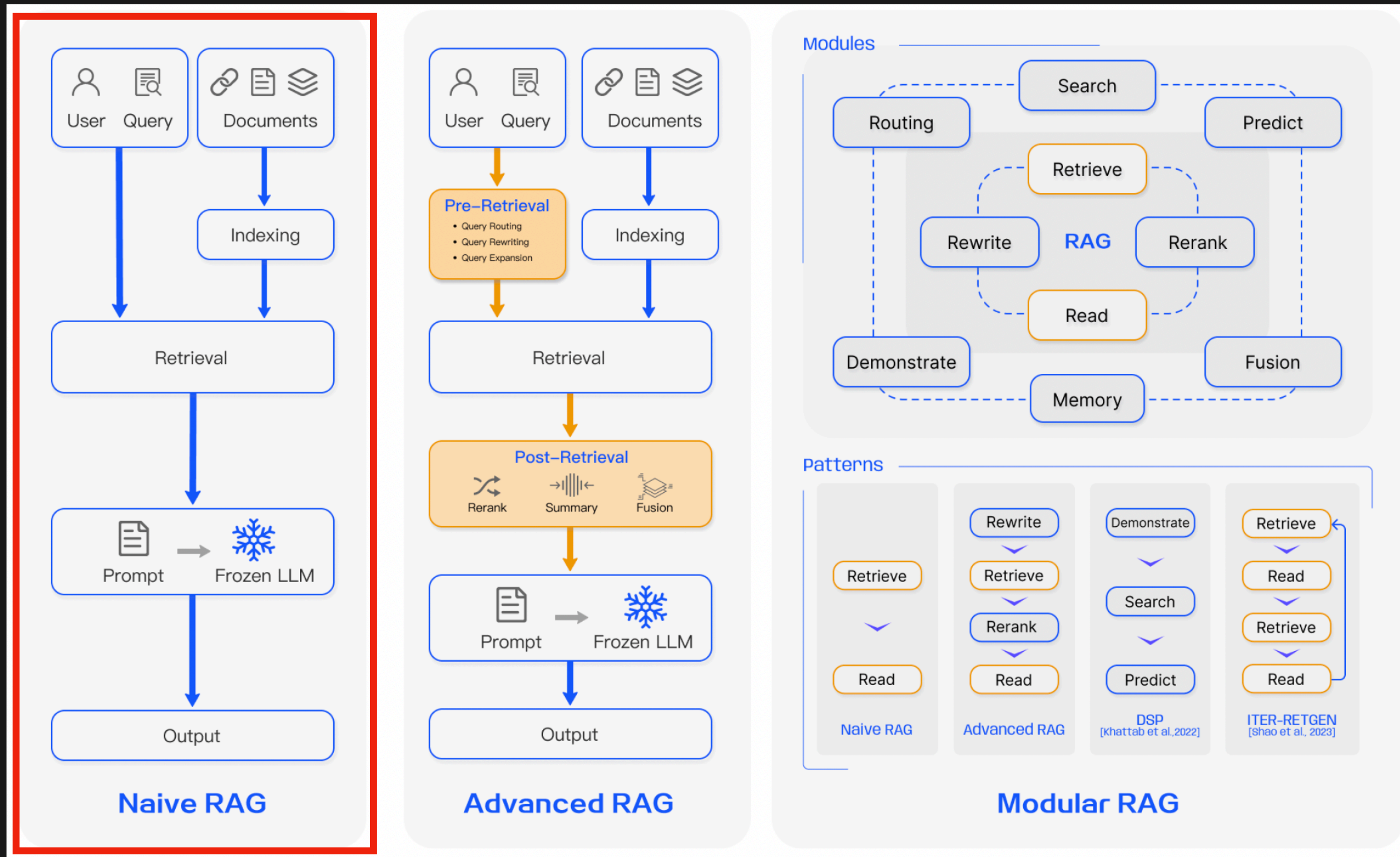
<https://arxiv.org/abs/2005.11401>

세 가지 RAG 방법론



<https://arxiv.org/pdf/2312.10997>

세 가지 RAG 방법론



<https://arxiv.org/pdf/2312.10997>

Naive RAG의 한계점 - Retrieval



Low precision



Low recall

Naive RAG의 한계점 - Augmentation



일관된 정보 retrieval



Evaluating relevancy

Naive RAG의 한계점 - Generation

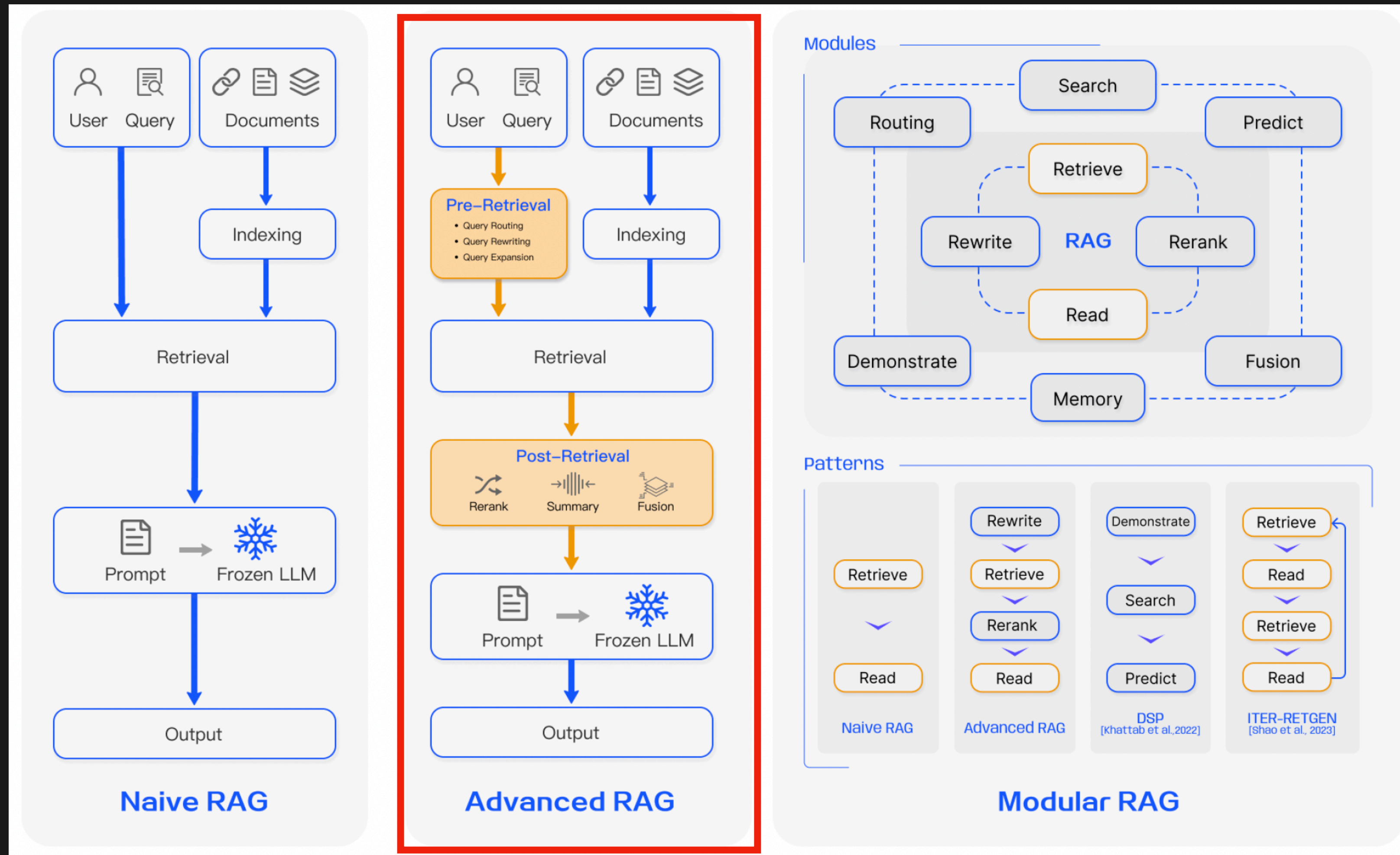


Hallucination



Irrelevance

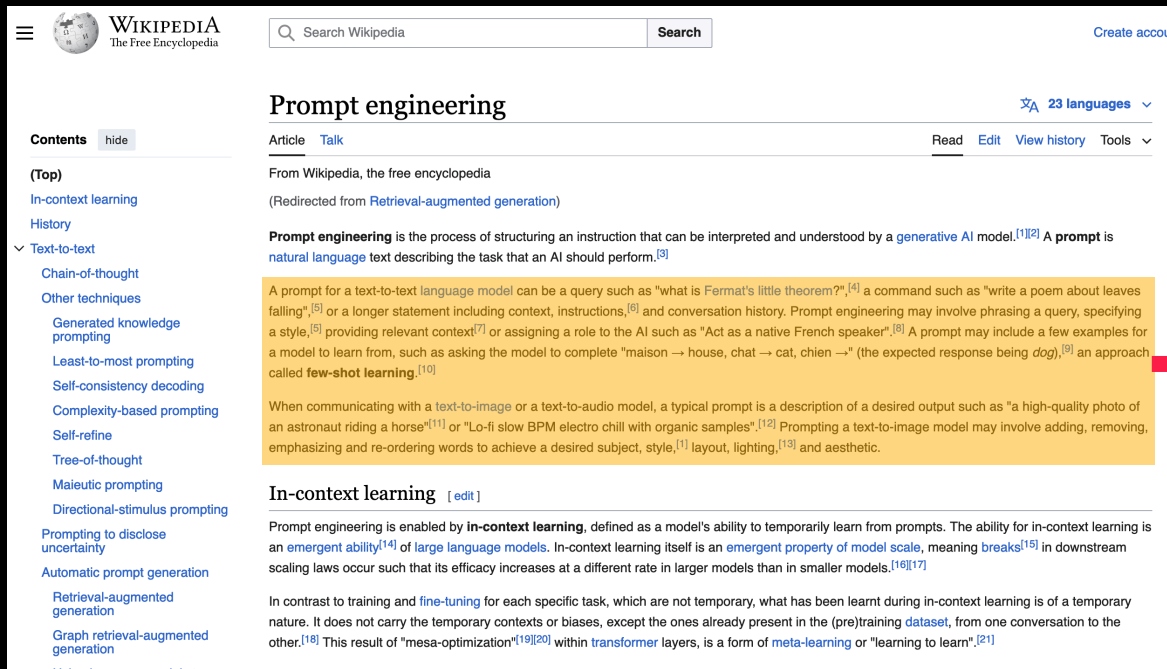
세 가지 RAG 방법론



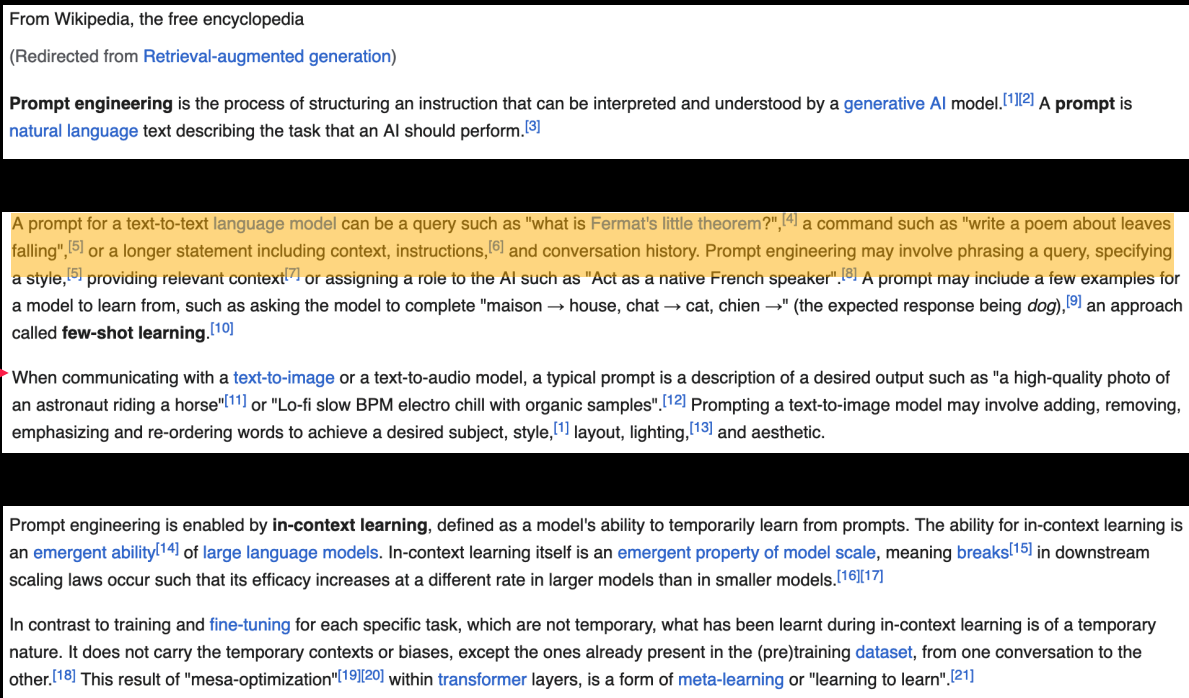
<https://arxiv.org/pdf/2312.10997>

RAG - pre retrieval, 인덱스 최적화

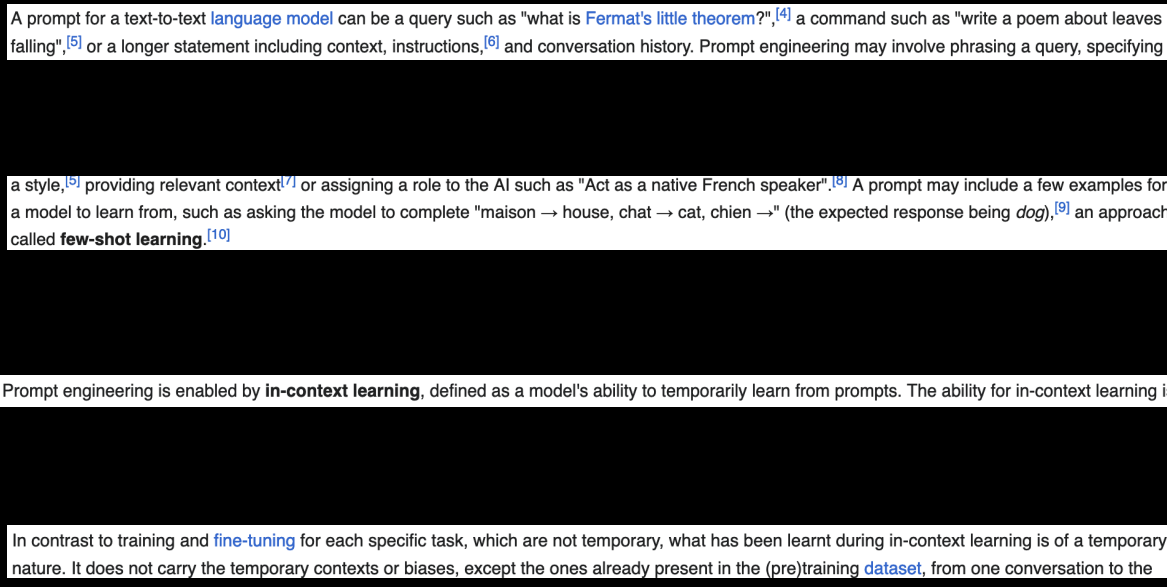
문서



문단

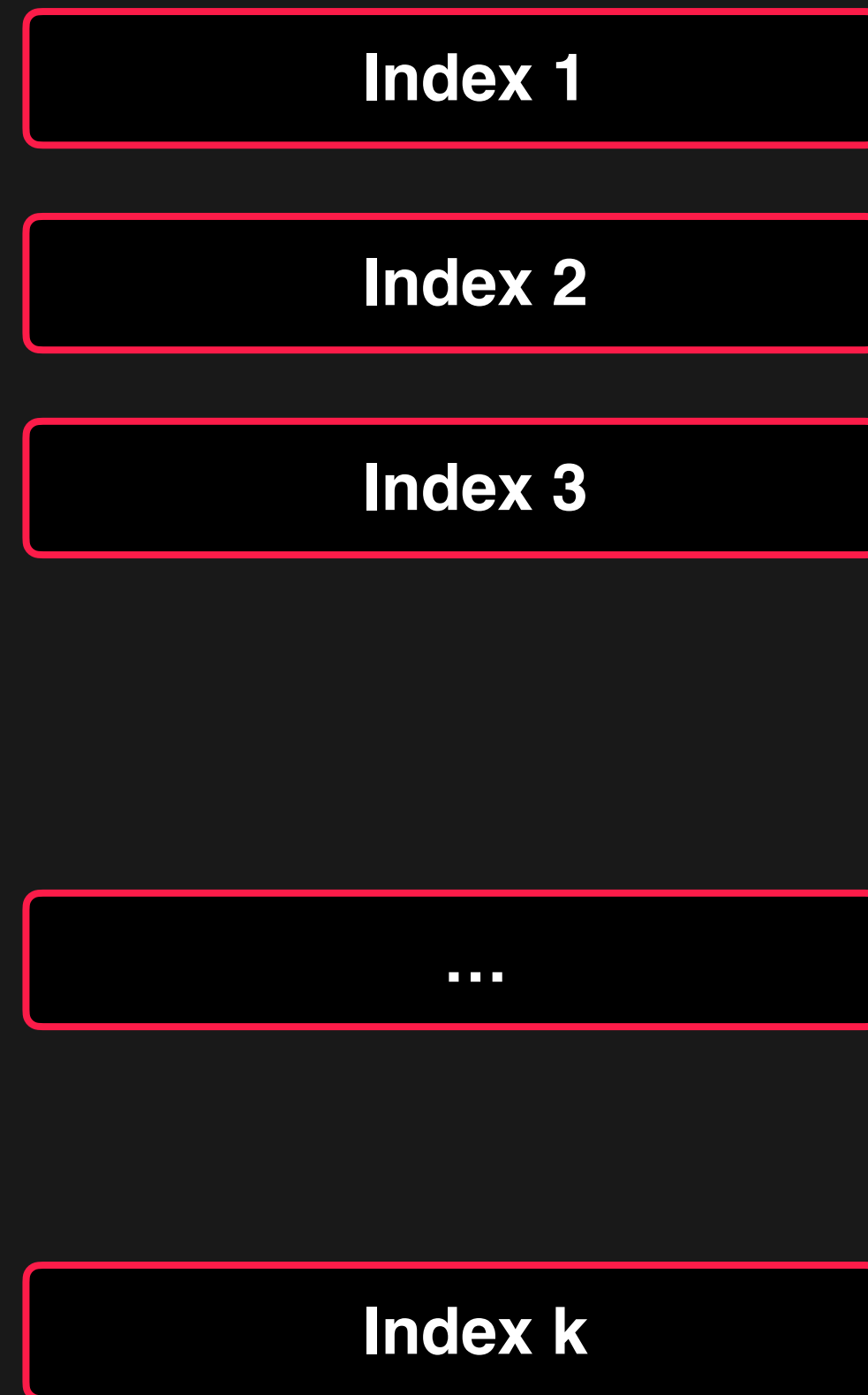


문장

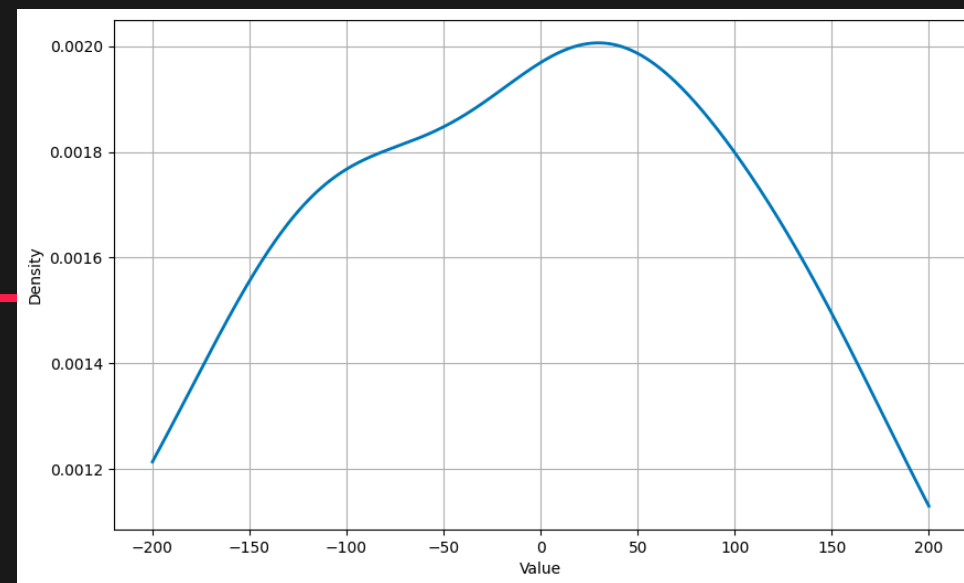


https://en.wikipedia.org/wiki/Prompt_engineering#Retrieval-augmented_generation

RAG - pre retrieval, 인덱스 최적화



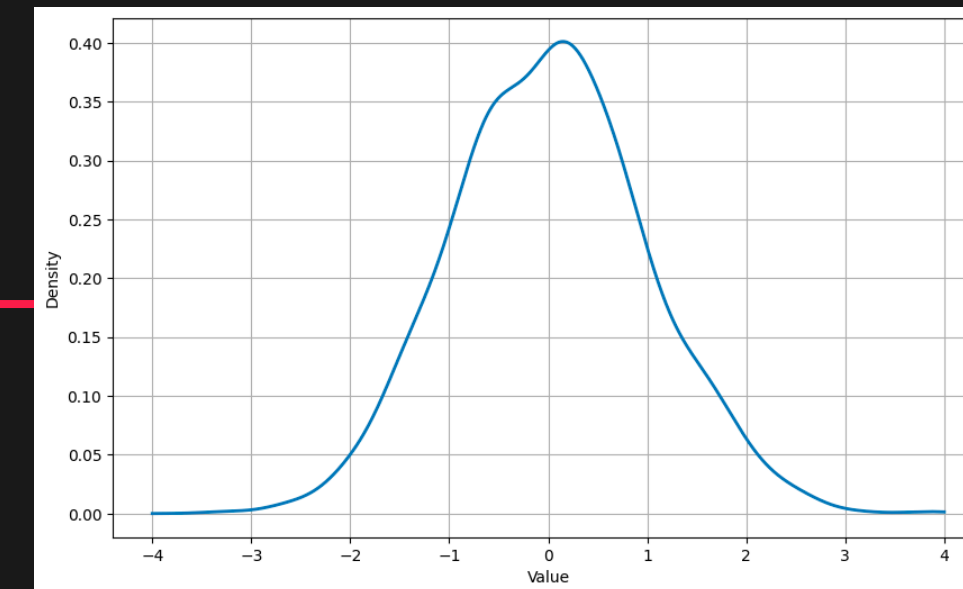
RAG - pre retrieval, 쿼리 최적화



Messy data



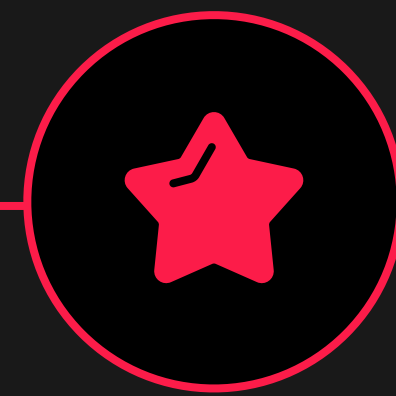
정규화



정규 분포



User query

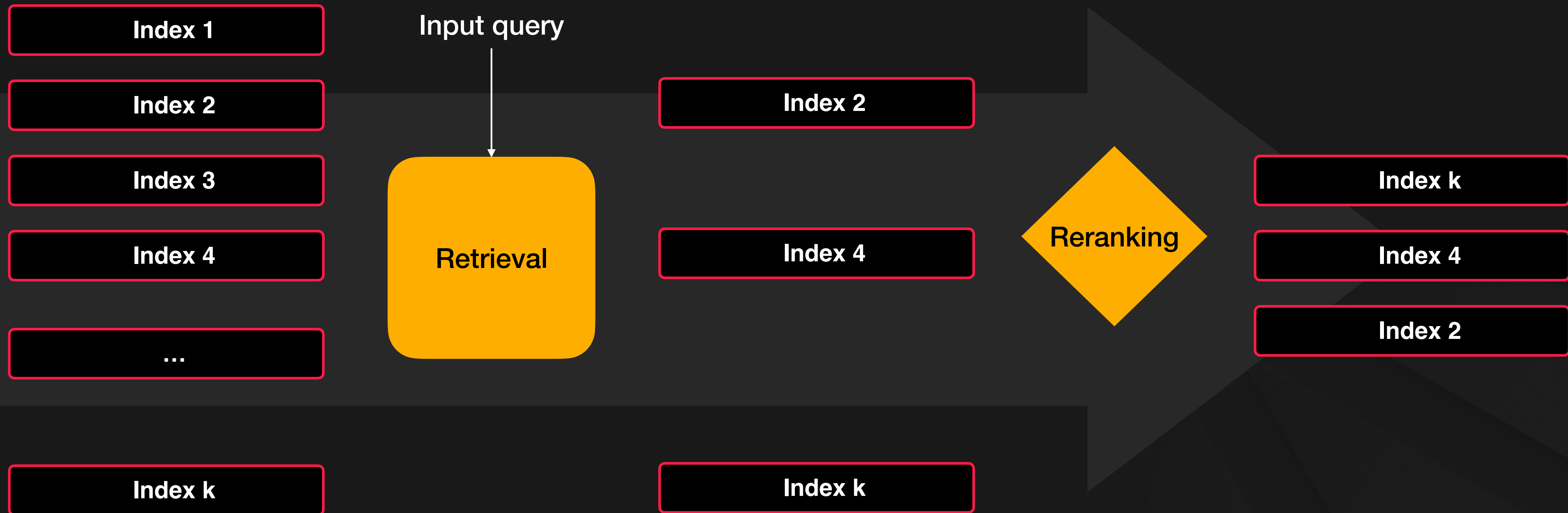


쿼리 최적화

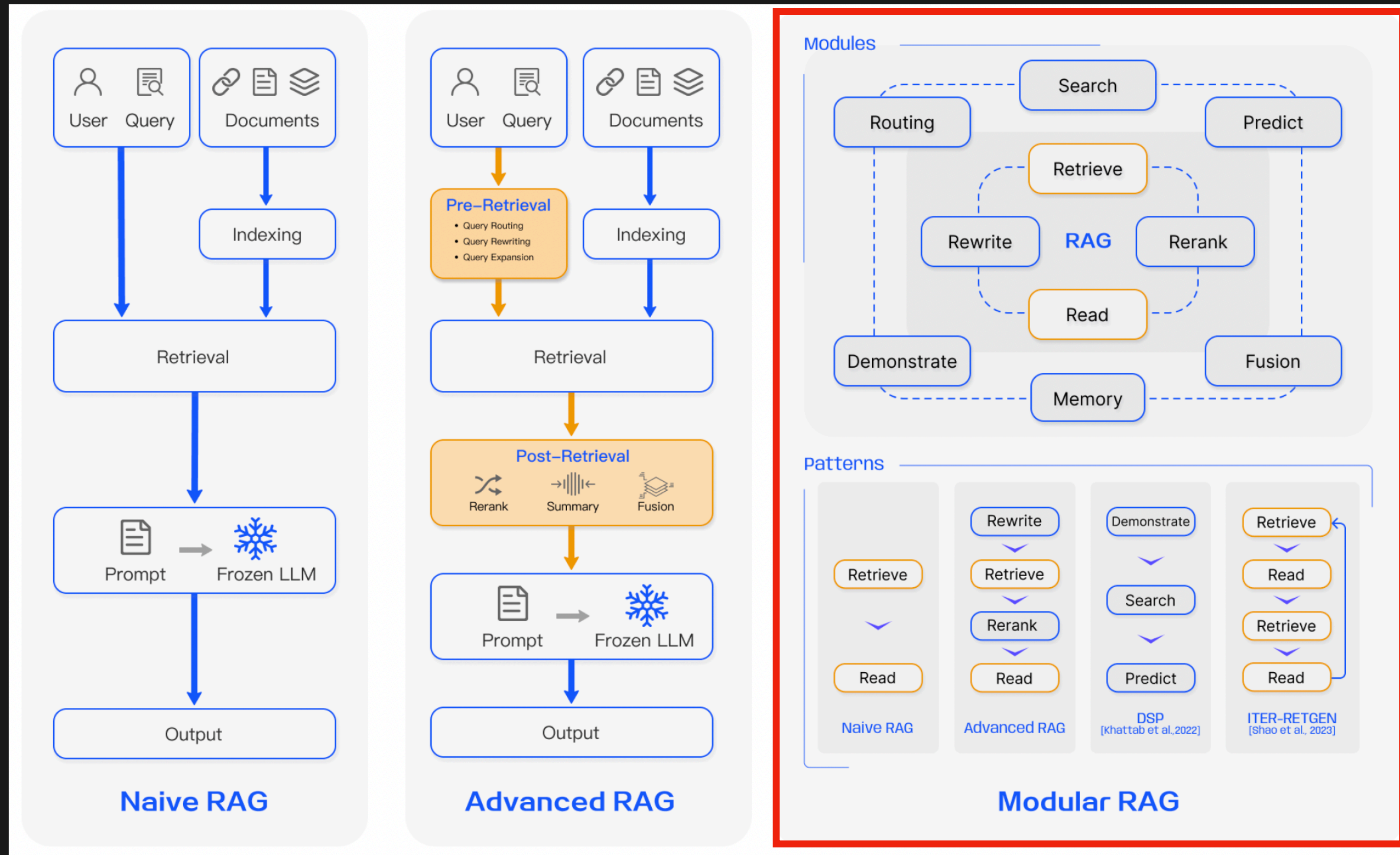


Refined query

RAG - post retrieval, reranking



세 가지 RAG 방법론



<https://arxiv.org/pdf/2312.10997>

Raptor RAG

Raptor Rag - 장문의 문서에 적합한 rag 방법론

단편적인 retrieval

글의 전체적인 구조 파악 필요
Long context → 비용 이슈

ex) How did Cinderella
reach her happy ending?



**Long text
RAG**

다양한 레벨의 사용자 query

“디테일 ↔ 전체 요약” 사이 어딘가

Ex) 논문의 다양한 섹션
introduction, methodology,
conclusion ...

Raptor Rag - methodology overview

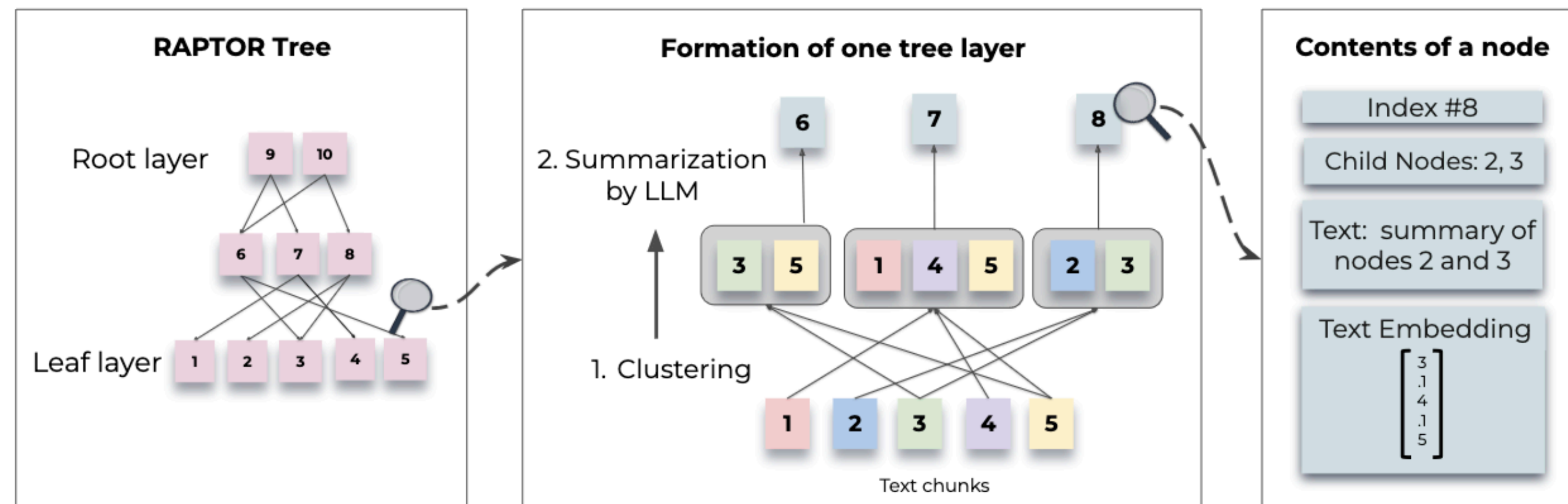


Figure 1: **Tree construction process:** RAPTOR recursively clusters chunks of text based on their vector embeddings and generates text summaries of those clusters, constructing a tree from the bottom up. Nodes clustered together are siblings; a parent node contains the text summary of that cluster.

Raptor Rag - methodology overview

Question: How does Cinderella find a happy ending?	
RAPTOR	The story of Cinderella involves her godmother, a fairy, who transforms a pumpkin into a grand coach with her wand and allows Cinderella to attend the ball. However, Cinderella must return home before the clock strikes eleven or her dress will turn back into rags... Cinderella impresses the Prince at the ball but leaves before he can find out who she is... The Prince searched for the owner of a lost glass slipper and found it belonged to Cinderella. She forgave her sisters and the Prince was glad to have found her.
DPR	the clock had struck Eleven... The Prince was very much surprised when he missed Cinderella again, and leaving the ball, went in search of her... Fairy touched Cinderella's rags, and they became rich satin robes, trimmed with point lace... Her old shoes became a charming pair of glass slippers, which shone like diamonds. "Now go to the ball, my love," she said, "and enjoy yourself. But remember, you must leave the room before the clock strikes eleven. If you do not your dress will return to its original rags."

Raptor Rag - methodology deep dive

Chunk text

100 token 단위로
문장이 끊어지지 않도록

Cluster

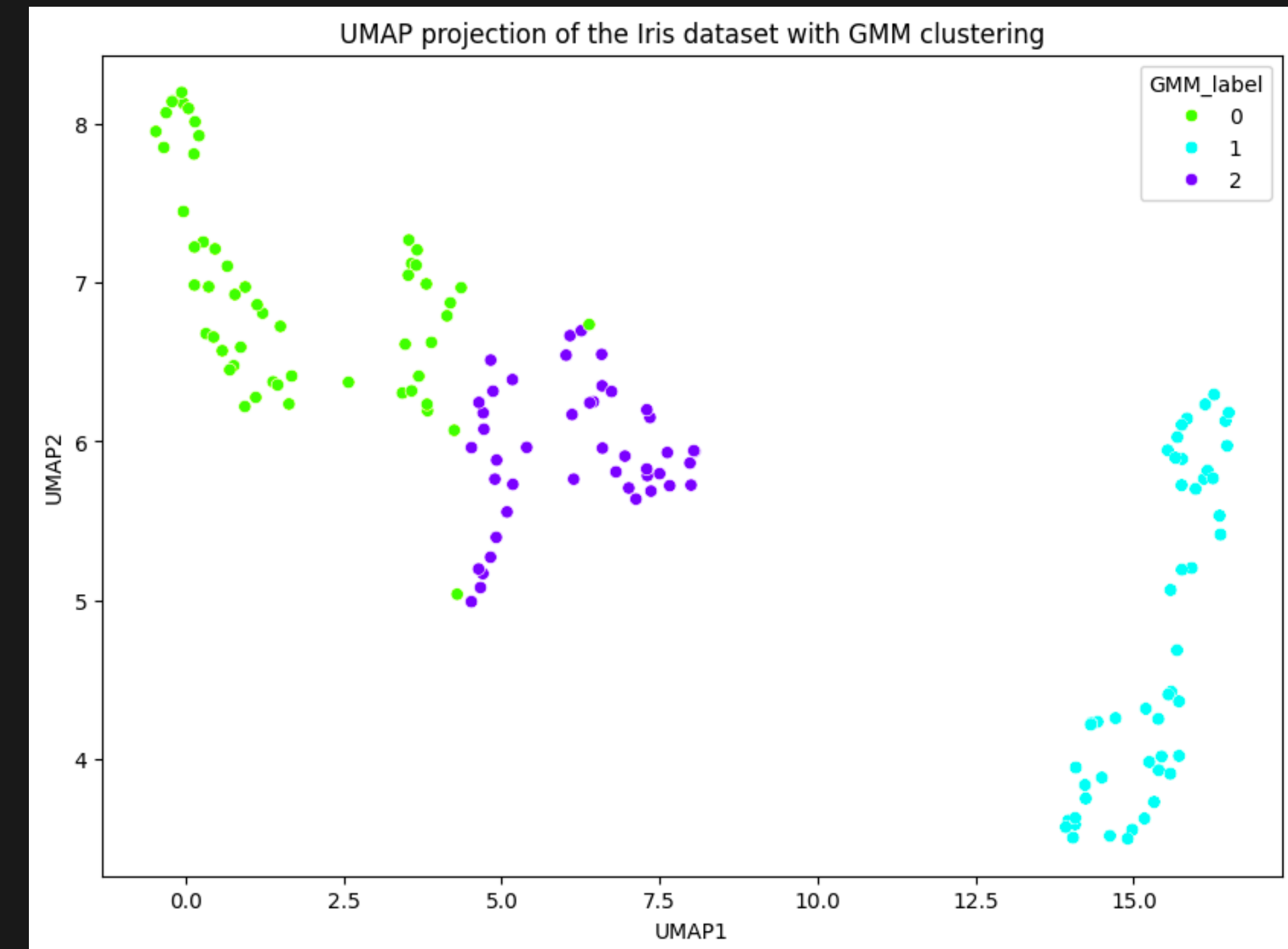
UMAP → BIC & GMM
(Soft clustering)

Summarize

Model based summarization

Retrieve

1. Tree traverse
2. Collapsed tree



<https://arxiv.org/pdf/2401.18059v1>

Raptor Rag - methodology deep dive

Chunk text

100 token 단위로
문장이 끊어지지 않도록

Cluster

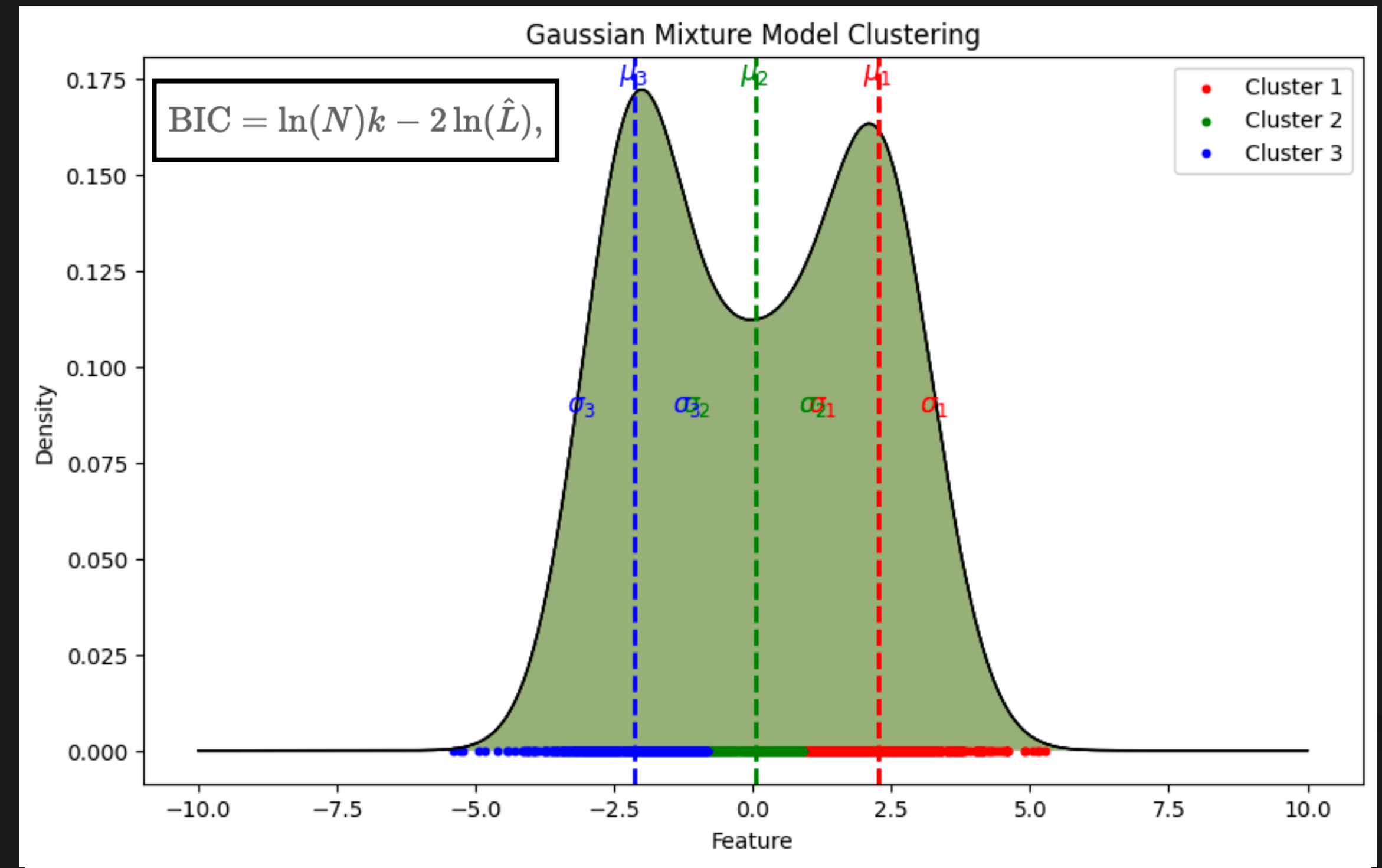
UMAP → BIC & GMM
(Soft clustering)

Summarize

Model based summarization

Retrieve

1. Tree traverse
2. Collapsed tree



<https://arxiv.org/pdf/2401.18059v1>

Raptor Rag - methodology deep dive

Chunk text

100 token 단위로
문장이 끊어지지 않도록

Cluster

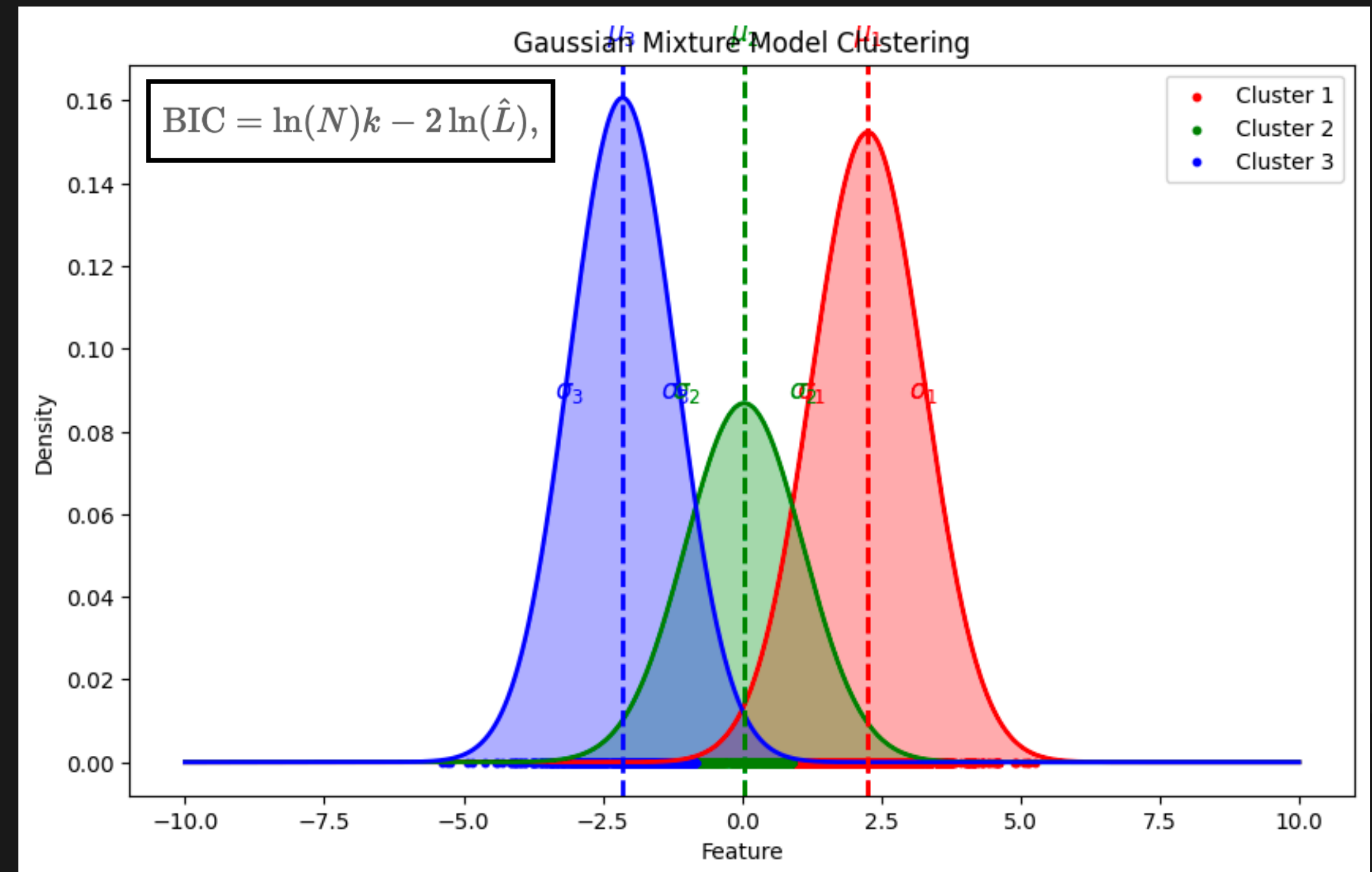
UMAP → BIC & GMM
(Soft clustering)

Summarize

Model based summarization

Retrieve

1. Tree traverse
2. Collapsed tree



<https://arxiv.org/pdf/2401.18059v1>

Raptor Rag - methodology deep dive

Chunk text

100 token 단위로
문장이 끊어지지 않도록

Cluster

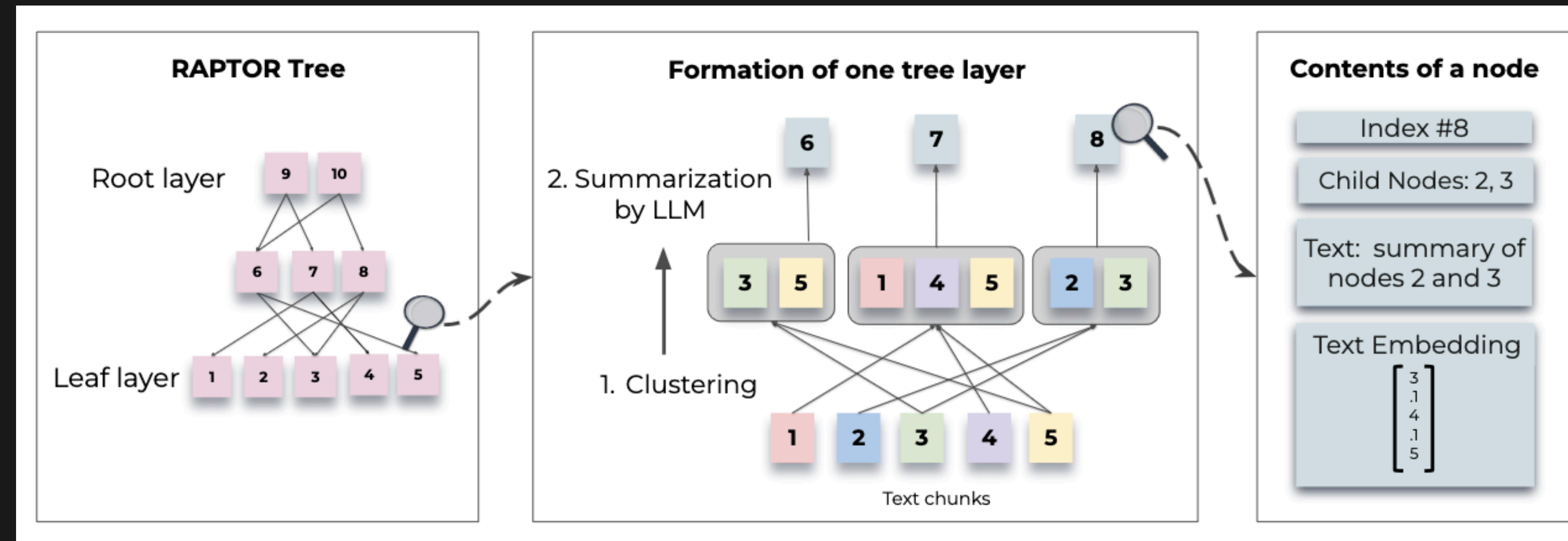
UMAP → BIC & GMM
(Soft clustering)

Summarize

Model based summarization

Retrieve

1. Tree traverse
2. Collapsed tree



<https://arxiv.org/pdf/2401.18059v1>

Raptor Rag - methodology deep dive

Chunk text

100 token 단위로
문장이 끊어지지 않도록

Cluster

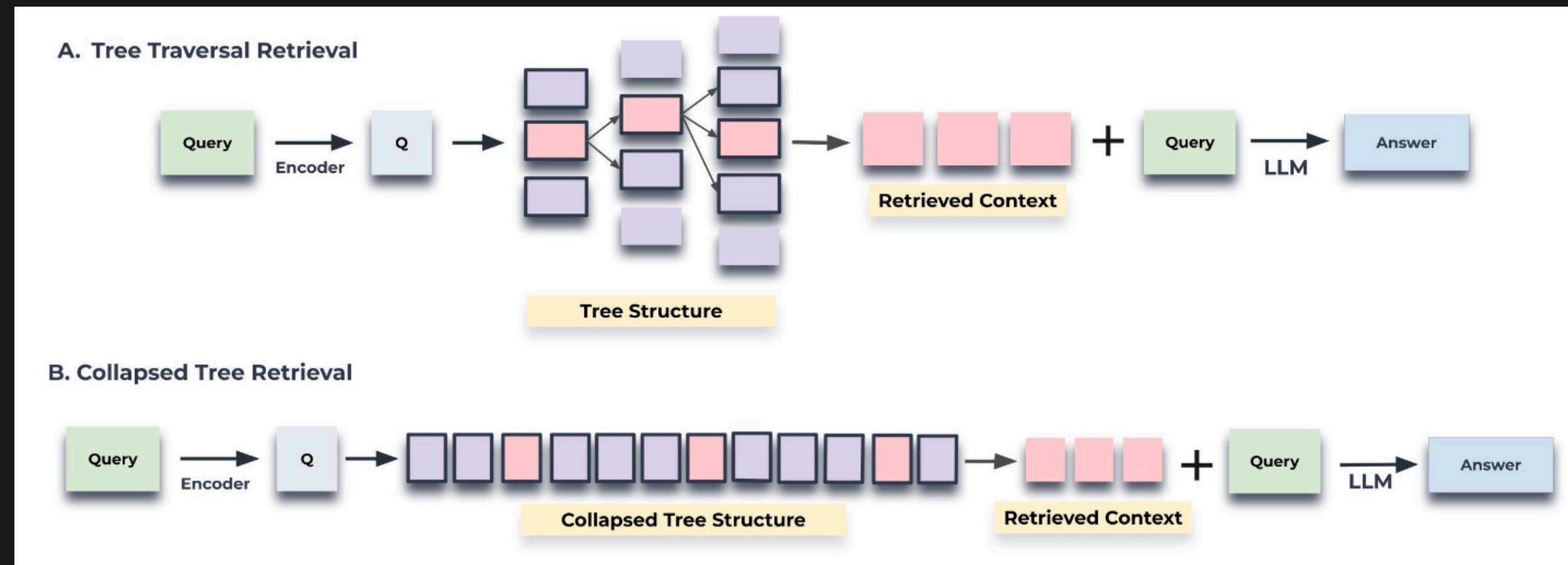
UMAP → BIC & GMM
(Soft clustering)

Summarize

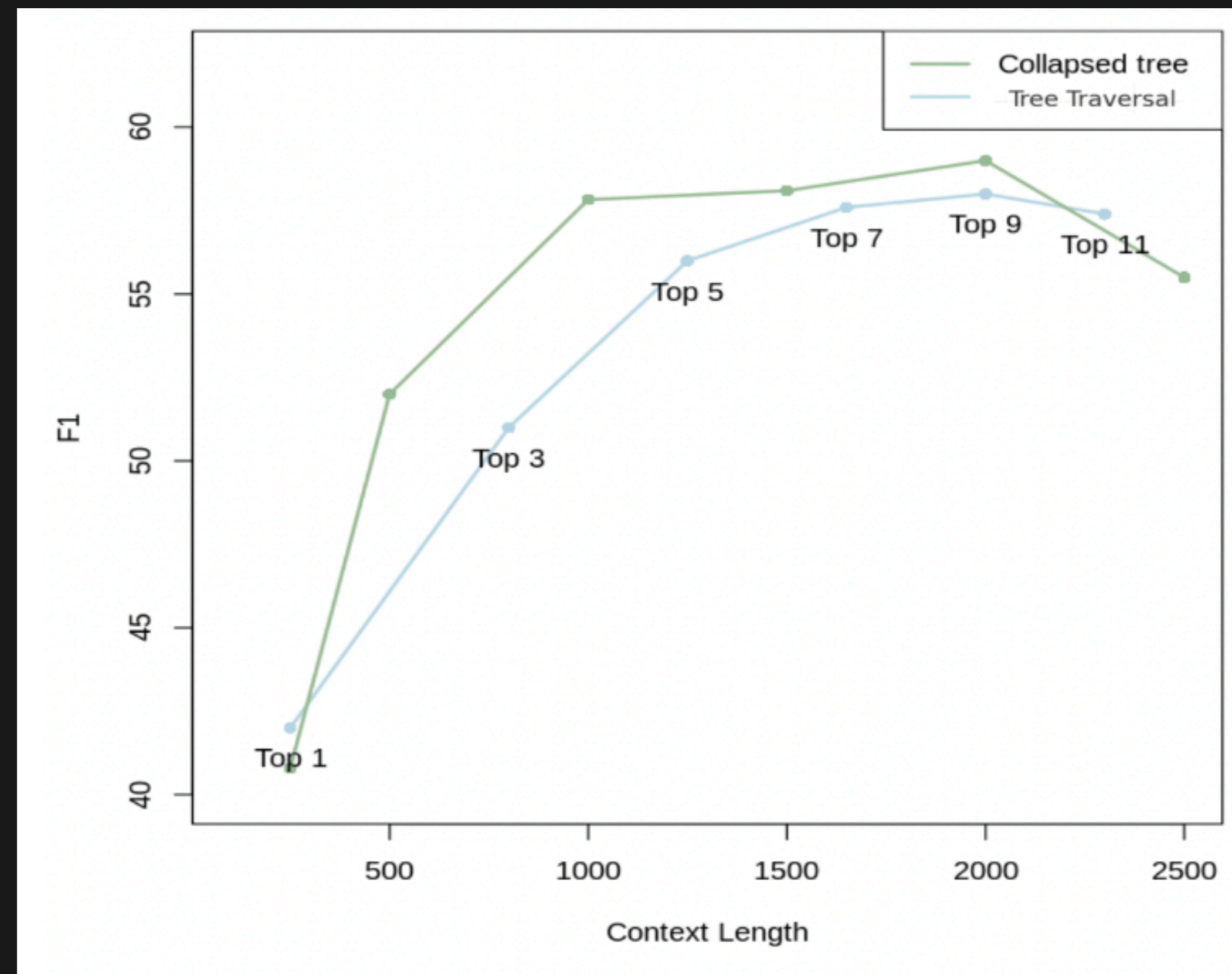
Model based summarization

Retrieve

1. Tree traverse
2. Collapsed tree



Raptor Rag - retrieval method



<https://arxiv.org/pdf/2401.18059v1>

Raptor Rag - performance

Model	Accuracy (QuALITY)	Answer F1 (QASPER)
SBERT with RAPTOR	56.6%	36.70%
SBERT without RAPTOR	54.9%	36.23%
BM25 with RAPTOR	52.1%	27.00%
BM25 without RAPTOR	49.9%	26.47%
DPR with RAPTOR	54.7%	32.23%
DPR without RAPTOR	53.1%	31.70%

QuALITY and QASPER Performance
With + Without RAPTOR

Retriever	GPT-3 F-1 Match	GPT-4 F-1 Match	UnifiedQA F-1 Match
Title + Abstract	25.2	22.2	17.5
BM25	46.6	50.2	26.4
DPR	51.3	53.0	32.1
RAPTOR	53.1	55.7	36.6

Controlled comparison of
F-1 scores on the QASPER dataset

Raptor Rag - performance

Dataset	DPR	SBERT	BM25
NarrativeQA	57.36%	36.78%	34.96%
Quality	32.28%	24.41%	32.36%
Qasper	22.93%	18.49%	22.76%

Non-leaf node / Retrieved node 개수

Raptor Rag - performance

Table 8: Performance of RAPTOR when querying different tree layers for Story 1 from the QuALITY dataset. Columns represent different starting points (highest layer) and rows represent different numbers of layers queried.

Layers Queried / Start Layer	Layer 0 (Leaf Nodes)	Layer 1	Layer 2
1 layer	57.9	57.8	57.9
2 layers	-	52.6	63.15
3 layers	-	-	73.68

활용 Layer에 따른 퍼포먼스

Raptor Rag - reference

Langchain - code snippet

- <https://github.com/langchain-ai/langchain/blob/master/cookbook/RAPTOR.ipynb>

Arxiv 논문

- <https://arxiv.org/pdf/2401.18059v1>

<https://arxiv.org/pdf/2401.18059v1>

Self RAG

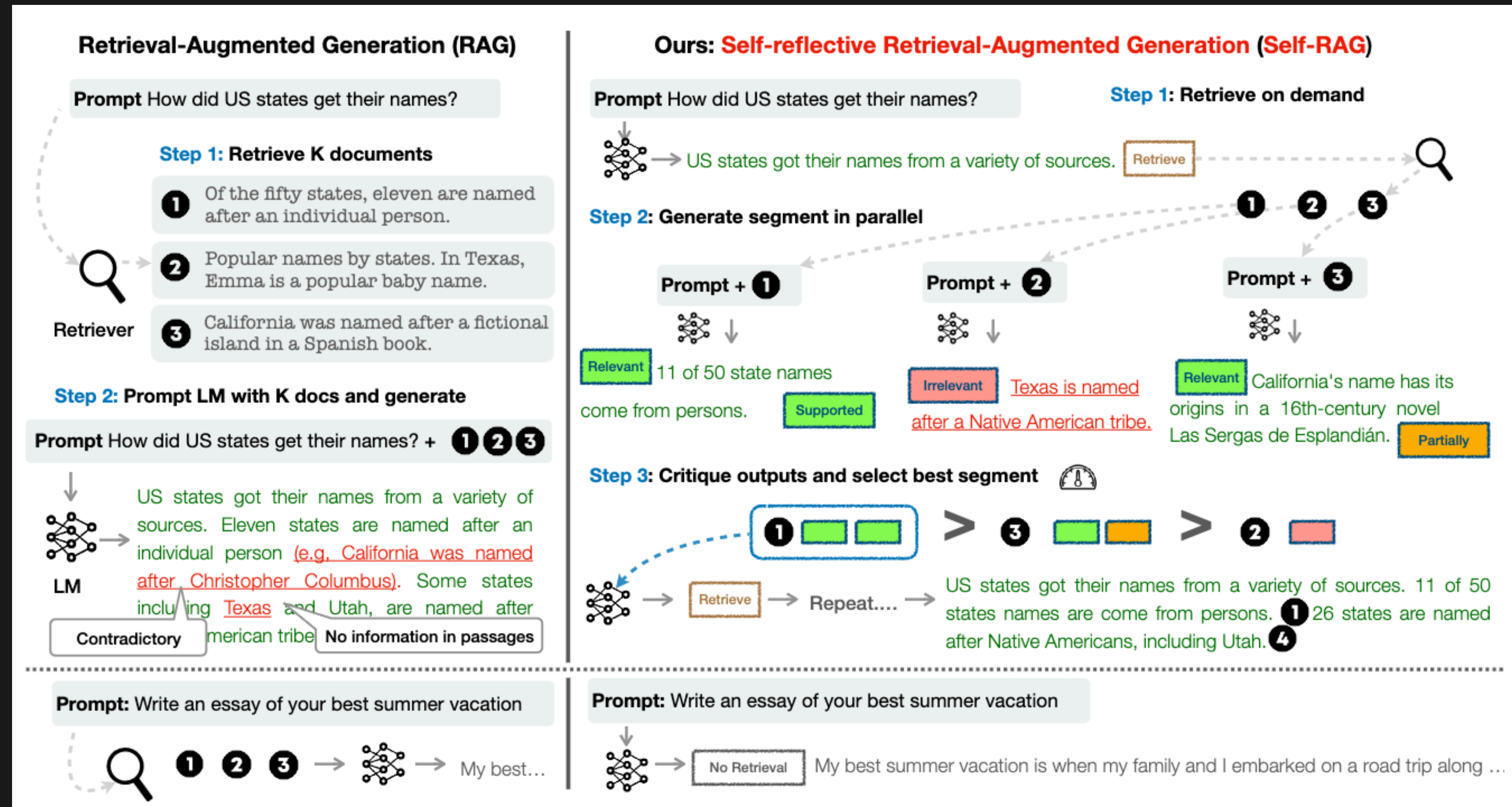
Self Rag - Overview

< 기존 RAG 방식의 문제점 >

- 사실적 부정확성
- 무분별한 문서 검색

< 해결 방법 >

- Self retrieval
- Self critique



Self Rag - key components

Type	Input	Output	Definitions
Retrieve	$x / x, y$	{yes, no, continue}	Decides when to retrieve with $\mathcal{R}$
ISREL	x, d	{ relevant , irrelevant}	d provides useful information to solve x .
ISSUP	x, d, y	{ fully supported , partially supported, no support}	All of the verification-worthy statement in y is supported by d .
ISUSE	x, y	{ 5 , 4, 3, 2, 1}	y is a useful response to x .

Self Rag - inference example

Input Synthesize a poem based on this prompt: “The sun is setting” (from `stanford_alpaca`)

Output [**Retrieve**=No]The stars begin to twinkle bright the trees, scattered throughout the night;
[**Retrieve**=No]The stars begin to twinkle bright the trees, scattered throughout the night [**Retrieve**=No] They
flicker like a silent dance, A peaceful moment in its trance. [**Retrieve**=No] They flicker like a silent dance, A
peaceful moment in its trance. [**Retrieve**=No] A chorus of crickets sing a lullaby, To every creature that roams
the sky [**ISUSE** =5]

Self Rag - inference example

Input when did walking dead season 7 come out (from nq)

Output [**Retrieve**=Yes]<p>The **Walking** Dead (season 7)–The Walking Dead (season 7) The seventh season of “The Walking Dead”, an American post-apocalyptic horror television series on AMC, premiered on October 23, 2016, and concluded on April 2, 2017, consisting of 16 episodes. Developed for television by Frank Darabont, the series is based on the eponymous series of comic books by Robert Kirkman, Tony Moore, and Charlie Adlard.
...<p>[**ISREL** =Relevant]October 23 , 2016[**ISSUP** =Fully Supported][**ISUSE** =5]

Self Rag - inference example

```
query_1 = "Leave odd one out: twitter, instagram, whatsapp."  
query_2 = "Can you tell me the difference between llamas and alpacas?"
```

Model prediction: Twitter, Instagram, and WhatsApp are all social media platforms. **[No Retrieval]** WhatsApp is the odd one out because it is a messaging app, while Twitter and # Instagram are primarily used for sharing photos and videos. **[Utility:5]** (this query doesn't require factual grounding; just skip retrieval and do normal instruction-following generation)

Model prediction: Sure! **[Retrieval]** <paragraph> ... (this query requires factual grounding, call a retriever)

https://huggingface.co/selfrag/selfrag_llama2_7b

Self Rag - inference example

Input Identify the two types of computer memory. (from gpt4_alpaca)

Output [Retrieve=Yes]<p>Computer memory Computer memory – In computing, memory refers to the computer hardware integrated circuits that store information for immediate use in a computer; it is synonymous with the term “primary storage”. Computer memory operates at a high speed, for example random-access memory (RAM), as a distinction from storage that provides slow-to-access information but offers higher capacities. If needed, contents of the computer memory can be transferred to secondary storage; a very common way of doing this is through a memory management technique called “virtual memory”. An archaic synonym for memory is store. The term “memory”, meaning “primary storage” or “main memory”, is<p>[IsREL=Relevant]Primary memory, also known as main memory, is the memory that the computer can directly access and manipulate. [IsSUP=Fully Supported] [Retrieve=Yes]<p>Computer memory—often associated with addressable semiconductor memory, i.e. integrated circuits consisting of silicon-based transistors, used for example as primary storage but also other purposes in computers and other digital electronic devices. There are two main kinds of semiconductor memory, volatile and non-volatile. Examples of non-volatile memory are flash memory (used as secondary memory) and ROM, PROM, EPROM and EEPROM memory (used for storing firmware such as BIOS). Examples of volatile memory are primary storage, which is typically dynamic random-access memory (DRAM), and fast CPU cache memory, which is typically static random-access memory (SRAM) that is fast but energy-consuming, offering lower memory<p>[IsREL=Relevant]Primary memory can be accessed quickly, but it is volatile, meaning that the data is lost if the power is turned off.[IsSUP=Partially Supported]...[IsUSE:5]

Self Rag - how it works

Algorithm 1 SELF-RAG Inference

Require: Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \dots, d_N\}$

- 1: **Input:** input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
- 2: $\mathcal{M}$ predicts **Retrieve** given $(x, y_{<t})$
- 3: **if** **Retrieve** == Yes **then**
- 4: Retrieve relevant text passages $\mathbf{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ **Retrieve**
- 5: $\mathcal{M}$ predicts **ISREL** given x, d and y_t given $x, d, y_{<t}$ for each $d \in \mathbf{D}$ ▷ **Generate**
- 6: $\mathcal{M}$ predicts **ISSUP** and **ISUSE** given x, y_t, d for each $d \in \mathbf{D}$ ▷ **Critique**
- 7: Rank y_t based on **ISREL**, **ISSUP**, **ISUSE** ▷ Detailed in Section **3.3**
- 8: **else if** **Retrieve** == No **then**
- 9: $\mathcal{M}_{gen}$ predicts y_t given x ▷ **Generate**
- 10: $\mathcal{M}_{gen}$ predicts **ISUSE** given x, y_t ▷ **Critique**

Self Rag - Performance. Metrics

- (Acc) Accuracy
- (FS) Fact Score
- (Em) Strict Exact Match
- (Rg) Rouge (correctness)
- (Mau) MAUCE (fluency)
- (Pre) Precision
- (Rec) Recall

Table 2: Overall experiment results on six tasks. **Bold** numbers indicate the best performance among non-proprietary models, and **gray-colored bold** text indicates the best proprietary model when they outperforms all non-proprietary models. * indicates concurrent or recent results reported by concurrent work. – indicates numbers that are not reported by the original papers or are not applicable. Models are sorted based on scale. FS, em, rg, mau, prec, rec denote FactScore (factuality); str-em, rouge (correctness); MAUVE (fluency); citation precision and recall, respectively.

LM	Short-form		Closed-set		Long-form generations (with citations)					
	PopQA (acc)	TQA (acc)	Pub (acc)	ARC (acc)	Bio (FS)	(em)	(rg)	ASQA (mau)	(pre)	(rec)
<i>LMs with proprietary data</i>										
Llama2-c _{13B}	20.0	59.3	49.4	38.4	55.9	22.4	29.6	28.6	–	–
Ret-Llama2-c _{13B}	51.8	59.8	52.1	37.9	79.9	32.8	34.8	43.8	19.8	36.1
ChatGPT	29.3	74.3	70.1	75.3	71.8	35.3	36.2	68.8	–	–
Ret-ChatGPT	50.8	65.7	54.7	75.3	–	40.7	39.9	79.7	65.1	76.6
Perplexity.ai	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines without retrieval</i>										
Llama2 _{7B}	14.7	30.5	34.2	21.8	44.5	7.9	15.3	19.0	–	–
Alpaca _{7B}	23.6	54.5	49.8	45.0	45.8	18.8	29.4	61.7	–	–
Llama2 _{13B}	14.7	38.5	29.4	29.4	53.4	7.2	12.4	16.0	–	–
Alpaca _{13B}	24.4	61.3	55.5	54.9	50.2	22.9	32.0	70.6	–	–
CoVE _{65B} *	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines with retrieval</i>										
Toolformer* _{6B}	–	48.8	–	–	–	–	–	–	–	–
Llama2 _{7B}	38.2	42.5	30.0	48.0	78.0	15.2	22.1	32.0	2.9	4.0
Alpaca _{7B}	46.7	64.1	40.2	48.0	76.6	30.9	33.3	57.9	5.5	7.2
Llama2-FT _{7B}	48.7	57.3	64.3	65.8	78.2	31.0	35.8	51.2	5.0	7.5
SAIL* _{7B}	–	–	69.2	48.4	–	–	–	–	–	–
Llama2 _{13B}	45.7	47.0	30.2	26.0	77.5	16.3	20.5	24.7	2.3	3.6
Alpaca _{13B}	46.1	66.9	51.1	57.6	77.7	34.8	36.7	56.6	2.0	3.8
Our SELF-RAG _{7B}	54.9	66.4	72.4	67.3	81.2	30.0	35.7	74.3	66.9	67.8
Our SELF-RAG _{13B}	55.8	69.3	74.5	73.1	80.2	31.7	37.0	71.6	70.3	71.3

<https://arxiv.org/pdf/2310.11511>

Self Rag - Performance. Dataset

- Short form
 - Gold answers
- Closed set
- Long-form generation

Table 2: Overall experiment results on six tasks. **Bold** numbers indicate the best performance among non-proprietary models, and **gray-colored bold** text indicates the best proprietary model when they outperforms all non-proprietary models. * indicates concurrent or recent results reported by concurrent work. – indicates numbers that are not reported by the original papers or are not applicable. Models are sorted based on scale. FS, em, rg, mau, prec, rec denote FactScore (factuality); str-em, rouge (correctness); MAUVE (fluency); citation precision and recall, respectively.

LM	Short-form		Closed-set		Long-form generations (with citations)					
	PopQA (acc)	TQA (acc)	Pub (acc)	ARC (acc)	Bio (FS)	(em)	(rg)	ASQA (mau)	(pre)	(rec)
<i>LMs with proprietary data</i>										
Llama2-c _{13B}	20.0	59.3	49.4	38.4	55.9	22.4	29.6	28.6	–	–
Ret-Llama2-c _{13B}	51.8	59.8	52.1	37.9	79.9	32.8	34.8	43.8	19.8	36.1
ChatGPT	29.3	74.3	70.1	75.3	71.8	35.3	36.2	68.8	–	–
Ret-ChatGPT	50.8	65.7	54.7	75.3	–	40.7	39.9	79.7	65.1	76.6
Perplexity.ai	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines without retrieval</i>										
Llama2 _{7B}	14.7	30.5	34.2	21.8	44.5	7.9	15.3	19.0	–	–
Alpaca _{7B}	23.6	54.5	49.8	45.0	45.8	18.8	29.4	61.7	–	–
Llama2 _{13B}	14.7	38.5	29.4	29.4	53.4	7.2	12.4	16.0	–	–
Alpaca _{13B}	24.4	61.3	55.5	54.9	50.2	22.9	32.0	70.6	–	–
CoVE _{65B} *	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines with retrieval</i>										
Toolformer* _{6B}	–	48.8	–	–	–	–	–	–	–	–
Llama2 _{7B}	38.2	42.5	30.0	48.0	78.0	15.2	22.1	32.0	2.9	4.0
Alpaca _{7B}	46.7	64.1	40.2	48.0	76.6	30.9	33.3	57.9	5.5	7.2
Llama2-FT _{7B}	48.7	57.3	64.3	65.8	78.2	31.0	35.8	51.2	5.0	7.5
SAIL* _{7B}	–	–	69.2	48.4	–	–	–	–	–	–
Llama2 _{13B}	45.7	47.0	30.2	26.0	77.5	16.3	20.5	24.7	2.3	3.6
Alpaca _{13B}	46.1	66.9	51.1	57.6	77.7	34.8	36.7	56.6	2.0	3.8
Our SELF-RAG _{7B}	54.9	66.4	72.4	67.3	81.2	30.0	35.7	74.3	66.9	67.8
Our SELF-RAG _{13B}	55.8	69.3	74.5	73.1	80.2	31.7	37.0	71.6	70.3	71.3

<https://arxiv.org/pdf/2310.11511>

Self Rag - Performance. Dataset

- LMs with Proprietary data
- Baselines without retrieval
- Baselines with retrieval
- SELF-RAG

Table 2: Overall experiment results on six tasks. **Bold** numbers indicate the best performance among non-proprietary models, and **gray-colored bold** text indicates the best proprietary model when they outperforms all non-proprietary models. * indicates concurrent or recent results reported by concurrent work. – indicates numbers that are not reported by the original papers or are not applicable. Models are sorted based on scale. FS, em, rg, mau, prec, rec denote FactScore (factuality); str-em, rouge (correctness); MAUVE (fluency); citation precision and recall, respectively.

LM	Short-form		Closed-set		Long-form generations (with citations)					
	PopQA (acc)	TQA (acc)	Pub (acc)	ARC (acc)	Bio (FS)	(em)	(rg)	ASQA (mau)	(pre)	(rec)
<i>LMs with proprietary data</i>										
Llama2-c _{13B}	20.0	59.3	49.4	38.4	55.9	22.4	29.6	28.6	–	–
Ret-Llama2-c _{13B}	51.8	59.8	52.1	37.9	79.9	32.8	34.8	43.8	19.8	36.1
ChatGPT	29.3	74.3	70.1	75.3	71.8	35.3	36.2	68.8	–	–
Ret-ChatGPT	50.8	65.7	54.7	75.3	–	40.7	39.9	79.7	65.1	76.6
Perplexity.ai	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines without retrieval</i>										
Llama2 _{7B}	14.7	30.5	34.2	21.8	44.5	7.9	15.3	19.0	–	–
Alpaca _{7B}	23.6	54.5	49.8	45.0	45.8	18.8	29.4	61.7	–	–
Llama2 _{13B}	14.7	38.5	29.4	29.4	53.4	7.2	12.4	16.0	–	–
Alpaca _{13B}	24.4	61.3	55.5	54.9	50.2	22.9	32.0	70.6	–	–
CoVE _{65B} *	–	–	–	–	71.2	–	–	–	–	–
<i>Baselines with retrieval</i>										
Toolformer* _{6B}	–	48.8	–	–	–	–	–	–	–	–
Llama2 _{7B}	38.2	42.5	30.0	48.0	78.0	15.2	22.1	32.0	2.9	4.0
Alpaca _{7B}	46.7	64.1	40.2	48.0	76.6	30.9	33.3	57.9	5.5	7.2
Llama2-FT _{7B}	48.7	57.3	64.3	65.8	78.2	31.0	35.8	51.2	5.0	7.5
SAIL* _{7B}	–	–	69.2	48.4	–	–	–	–	–	–
Llama2 _{13B}	45.7	47.0	30.2	26.0	77.5	16.3	20.5	24.7	2.3	3.6
Alpaca _{13B}	46.1	66.9	51.1	57.6	77.7	34.8	36.7	56.6	2.0	3.8
Our SELF-RAG _{7B}	54.9	66.4	72.4	67.3	81.2	30.0	35.7	74.3	66.9	67.8
Our SELF-RAG _{13B}	55.8	69.3	74.5	73.1	80.2	31.7	37.0	71.6	70.3	71.3

<https://arxiv.org/pdf/2310.11511>

Self Rag - reference

Huggingface - model repo

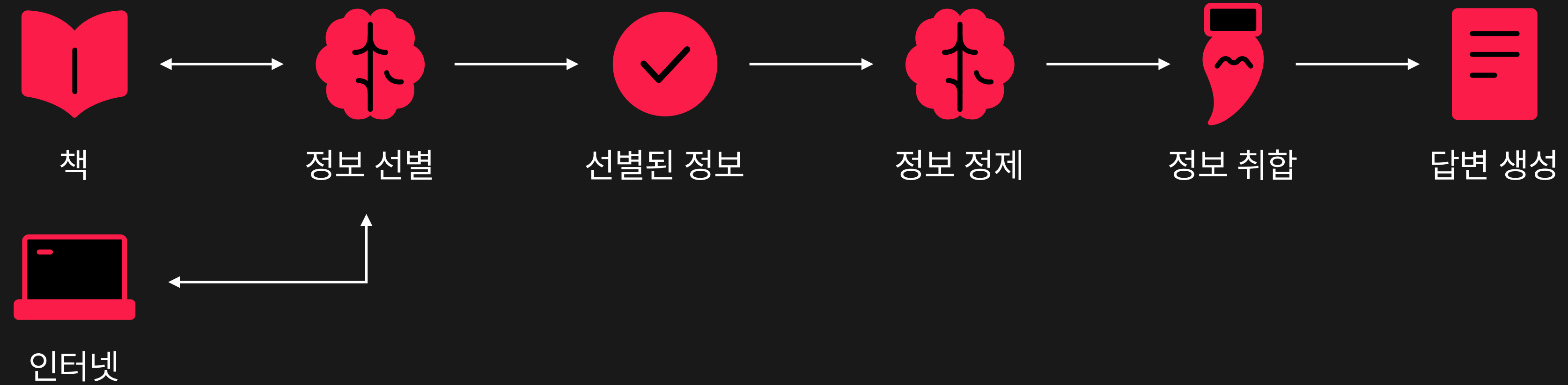
- https://huggingface.co/selfrag/selfrag_llama2_7b

Arxiv 논문

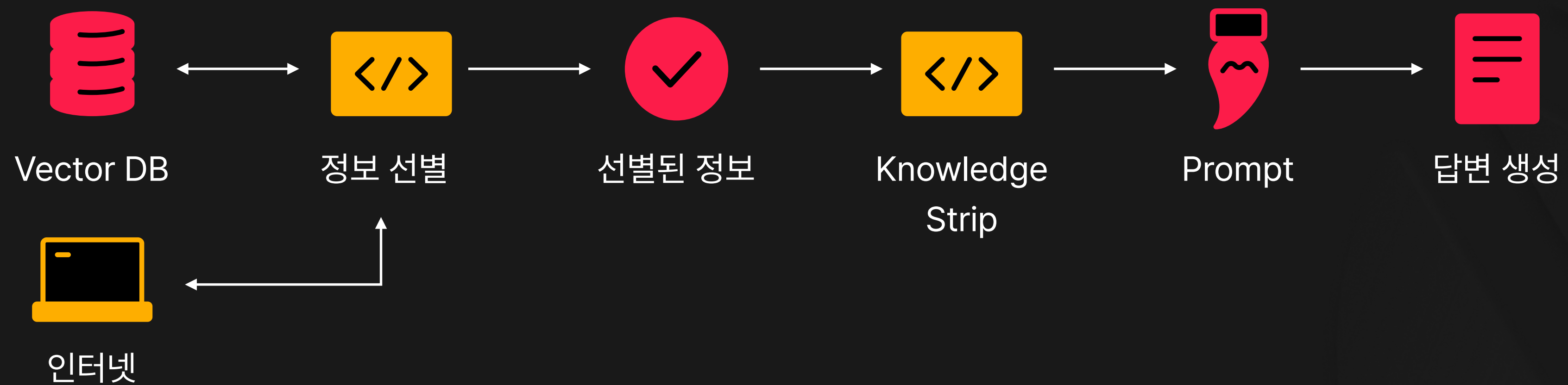
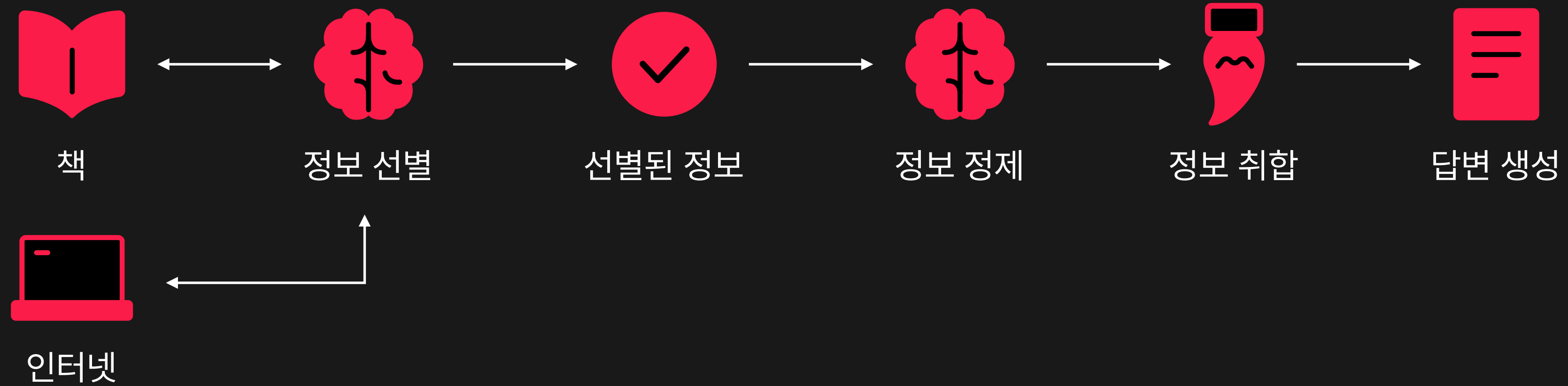
- <https://arxiv.org/pdf/2310.11511>

Corrective RAG

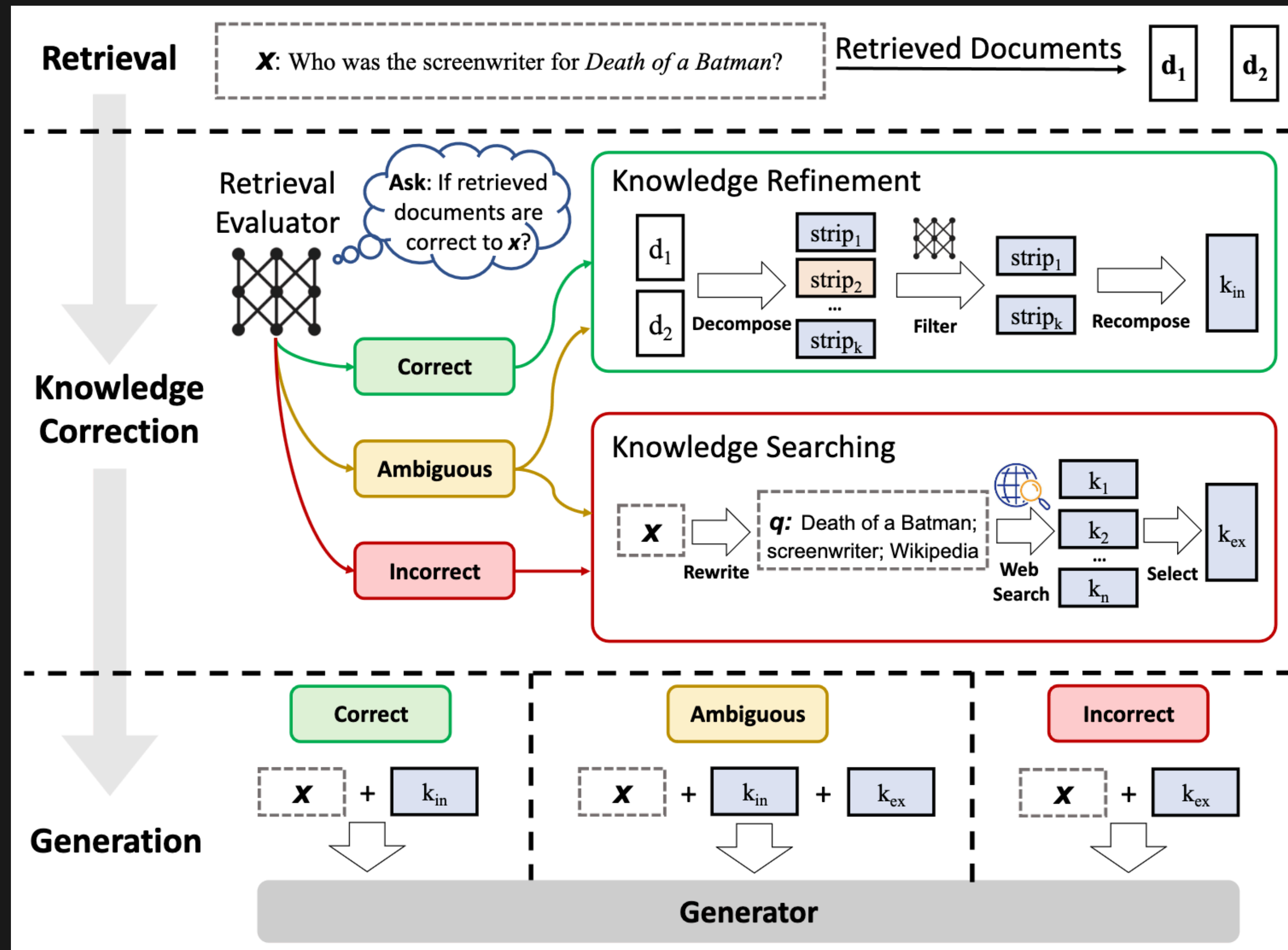
Corrective Rag - Overview



Corrective Rag - Overview



Corrective Rag - how it works

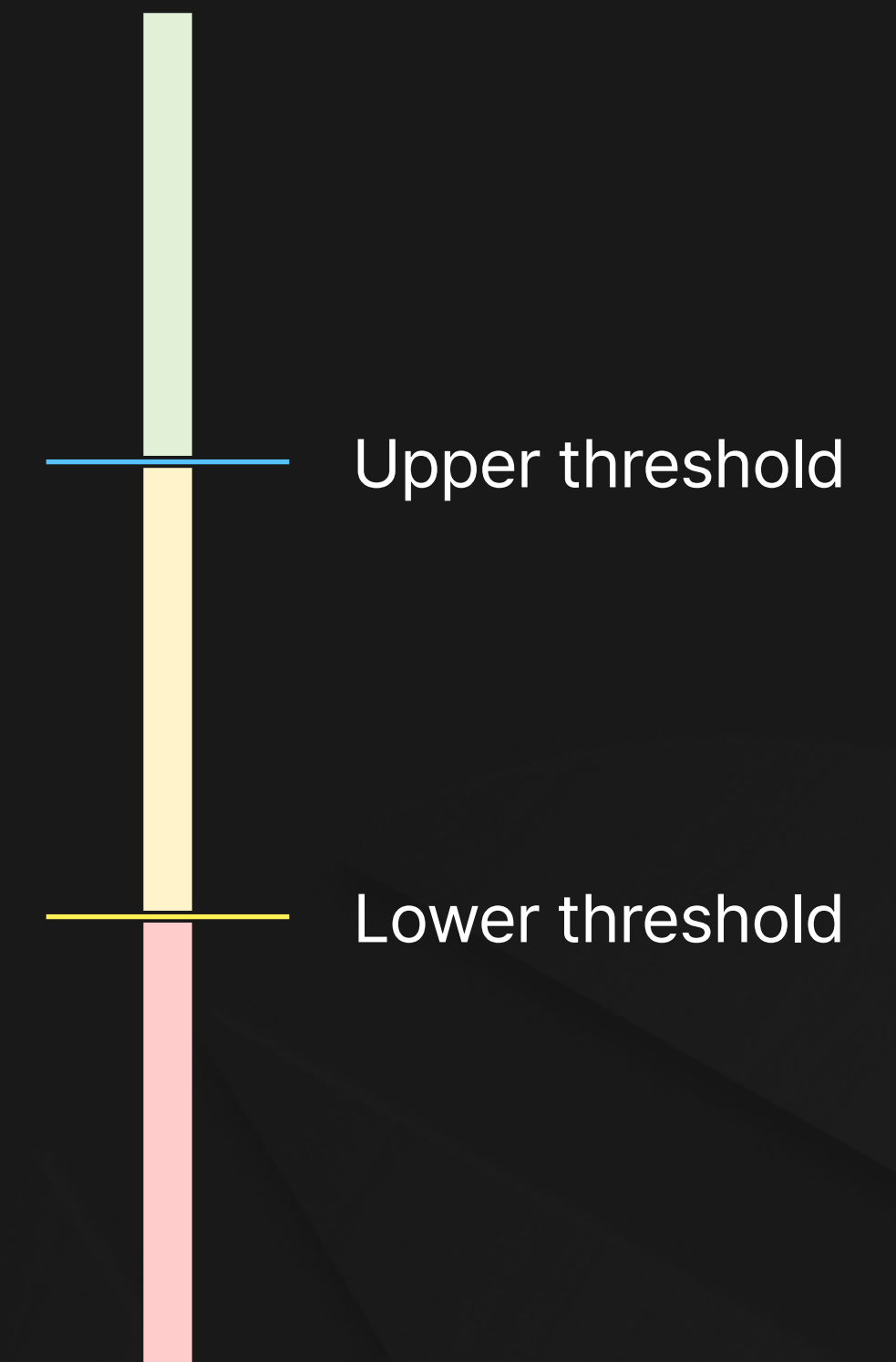


Corrective Rag - how it works

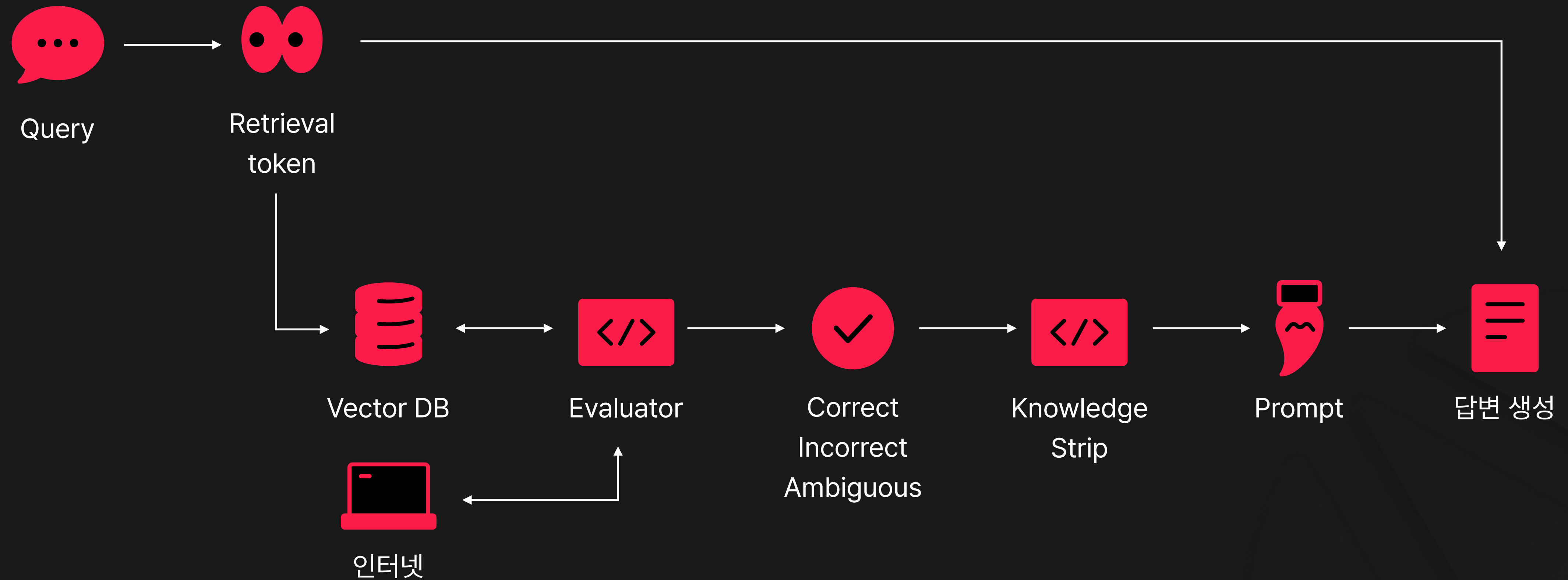
Algorithm 1: CRAG Inference

Require : E (Retrieval Evaluator), W (Query Rewriter), G (Generator)
Input : x (Input question), $D = \{d_1, d_2, \dots, d_k\}$ (Retrieved documents)
Output : y (Generated response)

- 1 $score_i = E$ evaluates the relevance of each pair (x, d_i) , $d_i \in D$
- 2 **Confidence** = Calculate and give a final judgment based on $\{score_1, score_2, \dots, score_k\}$
 // **Confidence** has 3 optional values: [CORRECT], [INCORRECT] or [AMBIGUOUS]
- 3 **if** $Confidence == [CORRECT]$ **then**
- 4 Internal_Knowledge = Knowledge_Refine(x, D)
- 5 $k = \text{Internal_Knowledge}$
- 6 **else if** $Confidence == [INCORRECT]$ **then**
- 7 External_Knowledge = Web_Search(W Rewrites x for searching)
- 8 $k = \text{External_Knowledge}$
- 9 **else if** $Confidence == [AMBIGUOUS]$ **then**
- 10 Internal_Knowledge = Knowledge_Refine(x, D)
- 11 External_Knowledge = Web_Search(W Rewrites x for searching)
- 12 $k = \text{Internal_Knowledge} + \text{External_Knowledge}$
- 13 **end**
- 14 G predicts y given x and k



Corrective Rag - incorporation into self-rag



<https://arxiv.org/pdf/2401.15884>

Corrective Rag - incorporation into self-rag

01

효율성 증가

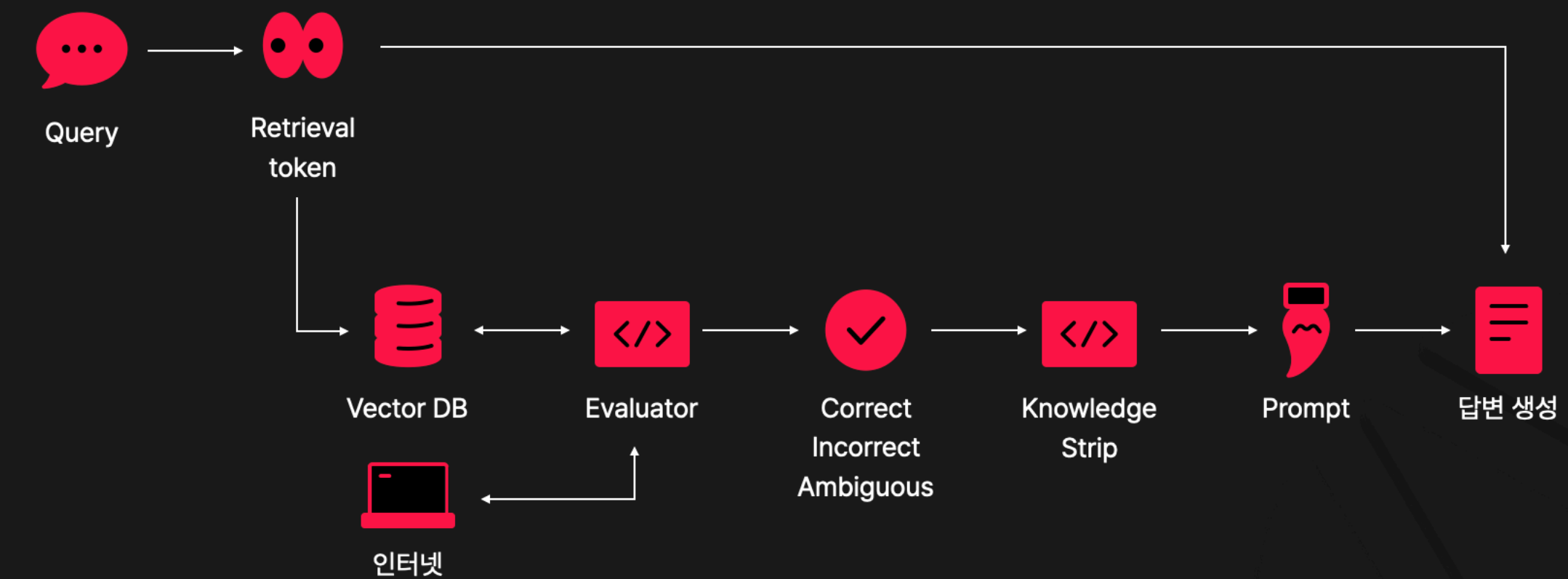
Inference (generation) time
Inference (generation) 리소스

02

Accuracy 증가

외부 knowledge 의존성 제한

- Retrieval 성능에 대해 민감 X
- 다양한 작업에 대한 일반화 능력 향상



<https://arxiv.org/pdf/2401.15884>

Corrective Rag - Performance

Method	PopQA (Accuracy)	Bio (FactScore)	Pub (Accuracy)	ARC (Accuracy)
<i>LMs trained with propriety data</i>				
LLaMA2-c _{13B}	20.0	55.9	49.4	38.4
Ret-LLaMA2-c _{13B}	51.8	79.9	52.1	37.9
ChatGPT	29.3	71.8	70.1	75.3
Ret-ChatGPT	50.8	-	54.7	75.3
Perplexity.ai	-	71.2	-	-
<i>Baselines without retrieval</i>				
LLaMA2 _{7B}	14.7	44.5	34.2	21.8
Alpaca _{7B}	23.6	45.8	49.8	45.0
LLaMA2 _{13B}	14.7	53.4	29.4	29.4
Alpaca _{13B}	24.4	50.2	55.5	54.9
CoVE _{65B}	-	71.2	-	-

<i>Baselines with retrieval</i>				
LLaMA2 _{7B}	38.2	78.0	30.0	48.0
Alpaca _{7B}	46.7	76.6	40.2	48.0
SAIL	-	-	69.2	48.4
LLaMA2 _{13B}	45.7	77.5	30.2	26.0
Alpaca _{13B}	46.1	77.7	51.1	57.6
<i>LLaMA2-hf-7b</i>				
RAG	37.7	44.9	9.1	23.8
CRAG	39.8	47.7	9.1	25.8
Self-RAG*	29.0	32.2	0.7	23.9
Self-CRAG	49.0	69.1	0.6	27.9
<i>SelfRAG-LLaMA2-7b</i>				
RAG	40.3	59.2	39.0	46.7
CRAG	59.3	74.1	75.6	54.8
Self-RAG	54.9	81.2	72.4	67.3
Self-CRAG	61.8	86.2	74.8	67.2

Corrective Rag - reference

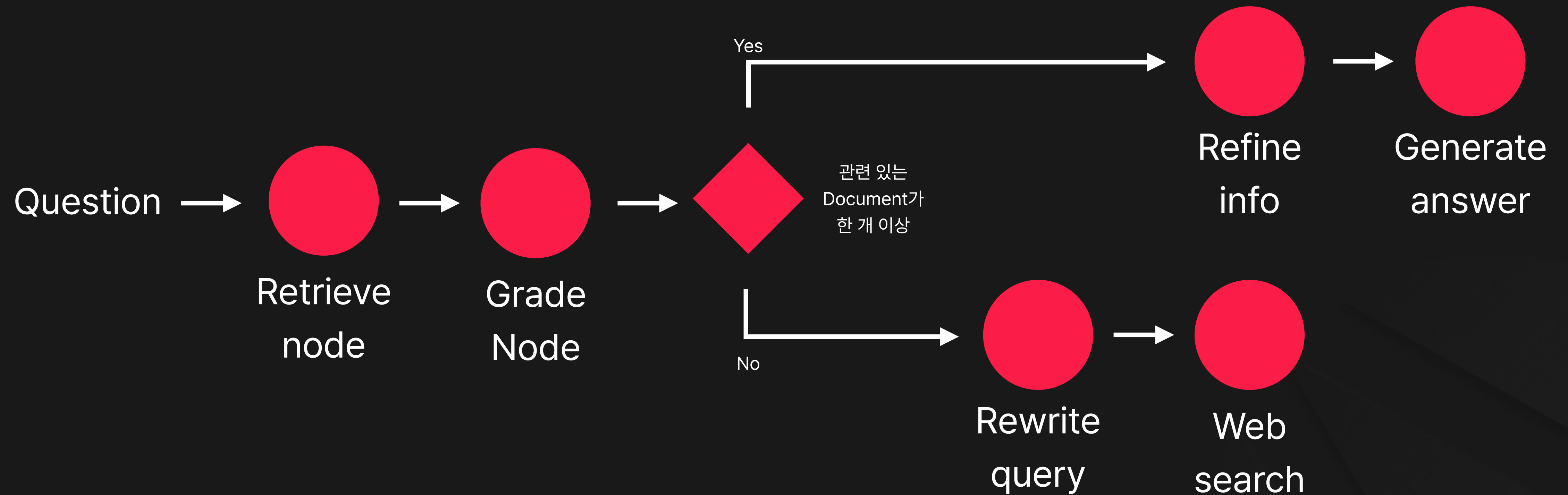
Langchain - code snippet

- <https://blog.langchain.dev/agent-rag-with-langgraph/>

Arxiv 논문

- <https://arxiv.org/pdf/2401.15884>

Corrective Rag - 실습



Tree RAG

Tree Rag - overview



인프라

- 보안을 위한 온프레미스 환경
- 제한된 컴퓨팅 리소스와 작은 training set



사용 문서

- 조직도
- 조직의 관리 매뉴얼

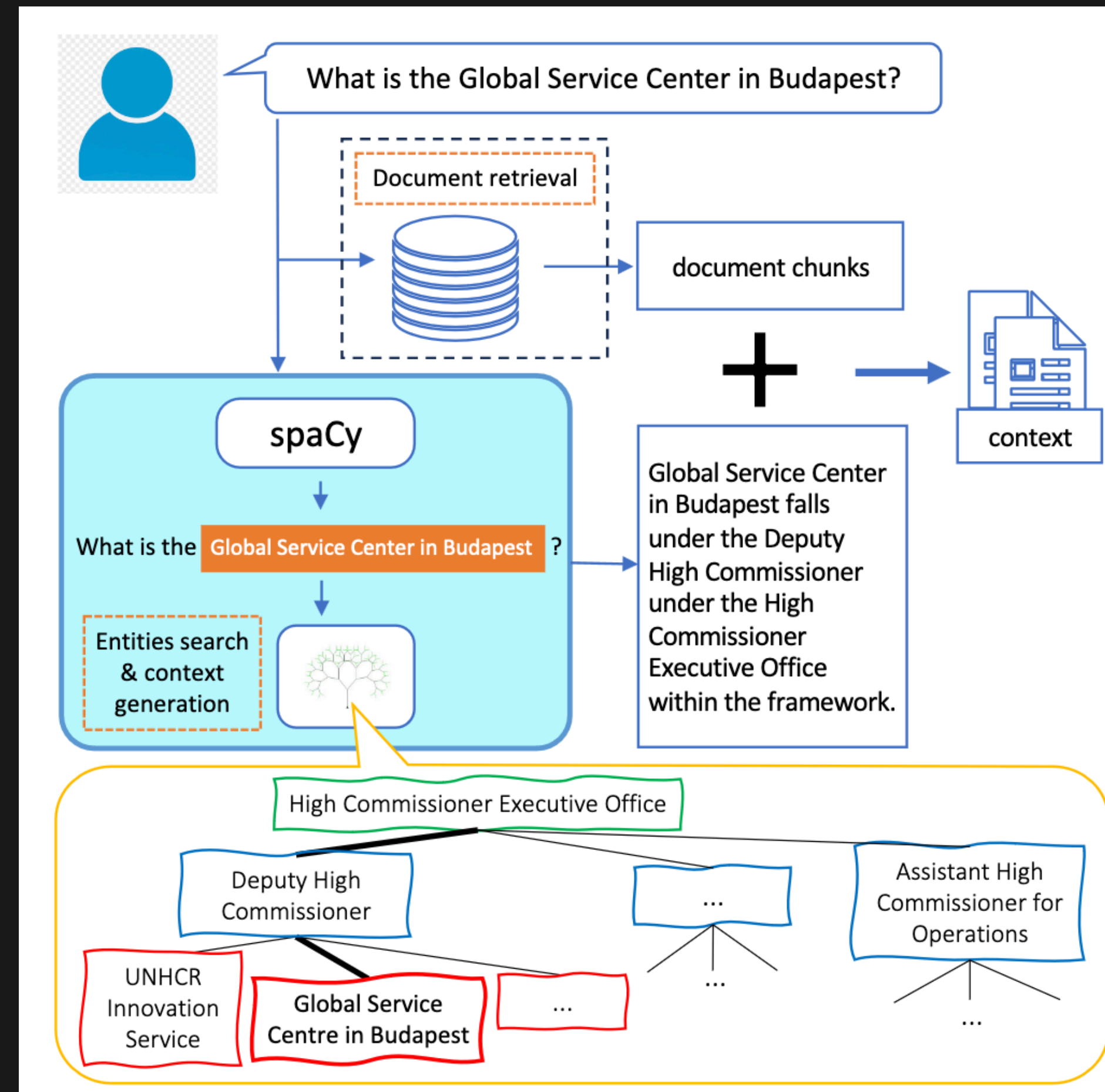


목표

- 문서 기반 질의 응답 시스템

- How does Giga³ plan to involve UNICEF and ITU counterparts in their strategy?
- Give examples of entities under HR Management?
- Who are the three broad categories of audiences that Giga will target in 2023?

Tree Rag - methodology



Tree Rag - methodology

Algorithm 1 High-level overview of a typical RAG system

Index Process

```
1: embeddings ← load("embedding_model")
2: doc ← load("file_name")
3: c ← chunker.chunk(doc)
4: ce ← embeddings.embed(c)
5: db ← index(ce)
```

Query Process

```
1: INITIALIZE sys_prompt ← "You are an AI...."
2: INITIALIZE model ← load("llm_model")
3: while TRUE do
4:   q ← get.user_query()
5:   qe ← embeddings.embed(q)
6:   chunks ← db.search(qe)
7:   context ← merge(chunks)
8:   prompt ← create_prompt(sys_prompt, q, context)
9:   answer ← model.generate(prompt)
10: end while
```

Algorithm 2 Our system (T-RAG)

Query Process

```
1: INITIALIZE sys_prompt ← "You are an AI...."
2: INITIALIZE model ← load("finetund_llm")
3: INITIALIZE tree ← build_tree("entities_file")
4: while TRUE do
5:   q ← get.user_query()
6:   qe ← embeddings.embed(q)
7:   chunks ← db.search(qe)
8:   entities ← parse_entities(q)
9:   if entities ≠ null then
10:    entities_info ← tree.search(entities)
11:    context ← merge(entities_info, chunks)
12:   else
13:    context ← merge(chunks)
14:   end if
15:   prompt ← create_prompt(sys_prompt, q, context)
16:   answer ← model.generate(prompt)
17: end while
```

Tree Rag - building the architecture



Fine-tune model

- Llama2 7B, QLora
- Chunk 된 문서마다 질문과 정답 생성
 - Data 1 :
True/False, summary, short answer
 - Data 2 :
User & AI 대화 생성
 - Data 3 :
사람이 생성한 질문을 활용한 few shot



활용 데이터

- 조직도 → knowledge graph
- 조직의 관리 매뉴얼 → vector DB

<https://arxiv.org/pdf/2402.07483>

Tree Rag - performance

Name	Question Set	N	C	CV	T	Perc.
RAG	set 1	17	9	0	9	52.9%
RAG	set 2	11	7	0	7	63.6%
RAG	set 3	9	4	1	5	55.6%
RAG	All	37	20	1	21	56.8%
Finetuned	set 1	17	11	1	12	70.6%
Finetuned	set 2	11	3	0	3	27.3%
Finetuned	set 3	9	5	0	5	55.6%
Finetuned	All	37	19	1	20	54.1%
T-RAG	set 1	17	9	4	13	76.5%
T-RAG	set 2	11	6	2	8	72.7%
T-RAG	set 3	9	6	0	6	66.7%
T-RAG	All	37	21	6	27	73.0%

Lesson

Base model 설정을 통한 퍼포먼스 비교

<https://arxiv.org/pdf/2402.07483>

Tree Rag - performance

Name	Simple (17 questions)	Complex (22 questions)
RAG	10 (58.5%)	11 (50%)
Finetuned	8 (47.1%)	8 (36.4%)
Finetuned + tree	17 (100%)	17 (77.3%)
T-RAG without tree	16 (94.1%)	10 (45.5%)
T-RAG	17 (100%)	15 (68.2%)

Lesson

주요 구성 요소 테스트를 통한 성능 향상 수치화

<https://arxiv.org/pdf/2402.07483>

Tree Rag - needle in a haystack

Jack Sparrow was a legendary pirate of the Seven Seas and the irreverent trickster of the Caribbean. ... into account, and was the quickest to seize the moment and make it his own; whether

We performed a Needle in a Haystack test to further evaluate how the performance of the model is affected based on the ... chunks containing the relevant context for RAG) at different positions in the context surrounded by additional k unrelated context chunks.

Now one of the most infamous and notorious pirates of the Caribbean, Captain Jack Sparrow embarked on many adventures. He was captain of the Black Pearl for two years, during which ... help of Will Turner and Elizabeth Swann, Jack killed the cursed Captain Barbossa and retrieved

Position	T-RAG		RAG	
	<i>k</i> = 2	<i>k</i> = 10	<i>k</i> = 2	<i>k</i> = 10
Top	29 (74.4%)	27 (69.2%)	30 (76.9%)	22 (56.4%)
Mid	27 (69.2%)	25 (64.1%)	26 (66.7%)	13 (33.3%)
End	32 (82.1%)	28 (71.8%)	25 (64.1%)	18 (46.2%)

Tree Rag - response optimization

Table 5: Misalignment error: An example question and response generated by RAG with document chunks (no tree context) based on UNHCR entities. The entities in bold-red are actually under the *High Commissioner Executive Office* but incorrectly listed under the *Deputy High Commissioner*.

Question	Name some entities under Deputy high Commissioner and some under External Relations?
Response	Based on the provided context, the entities under the Deputy High Commissioner include: 1. Deputy High Commissioner 2. Inspector General Office 3. Policy Development & Evaluation Office And the entities under External Relations include: 1. Governance & Partnership 2. Inter-agency Coordination 3. Strategic Communications 4. Private Sector Fund Raising Service 5. Communications & Public Information

Lesson

정성적 분석을 통한 고도화 포인트 발굴

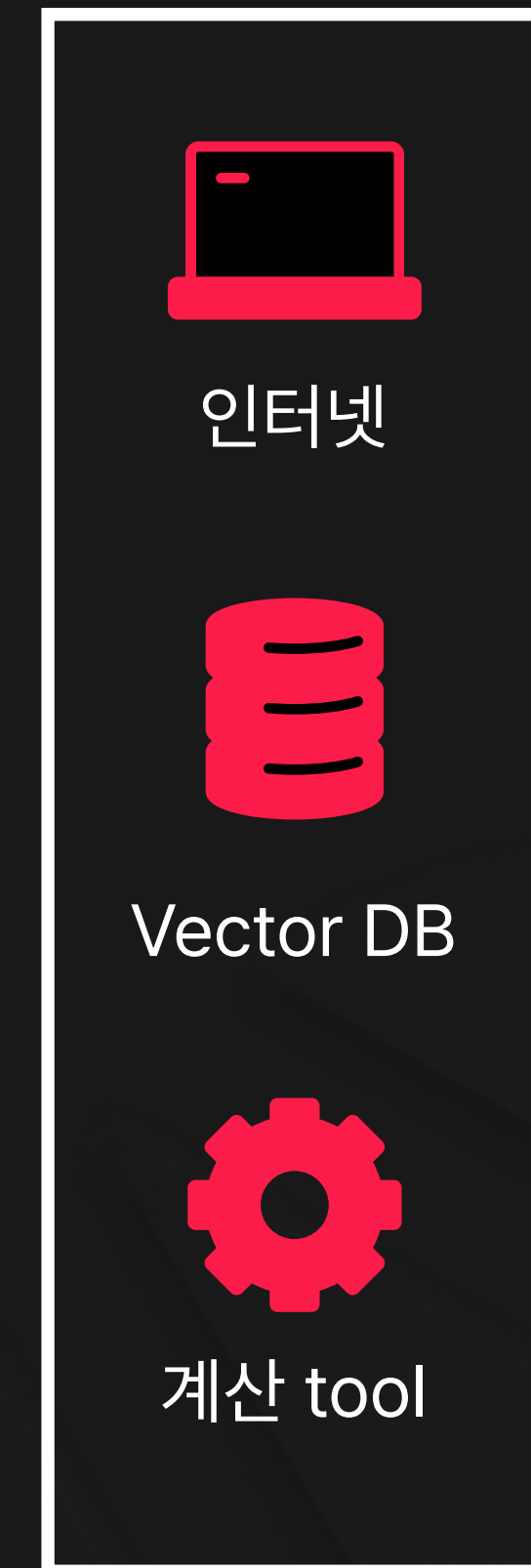
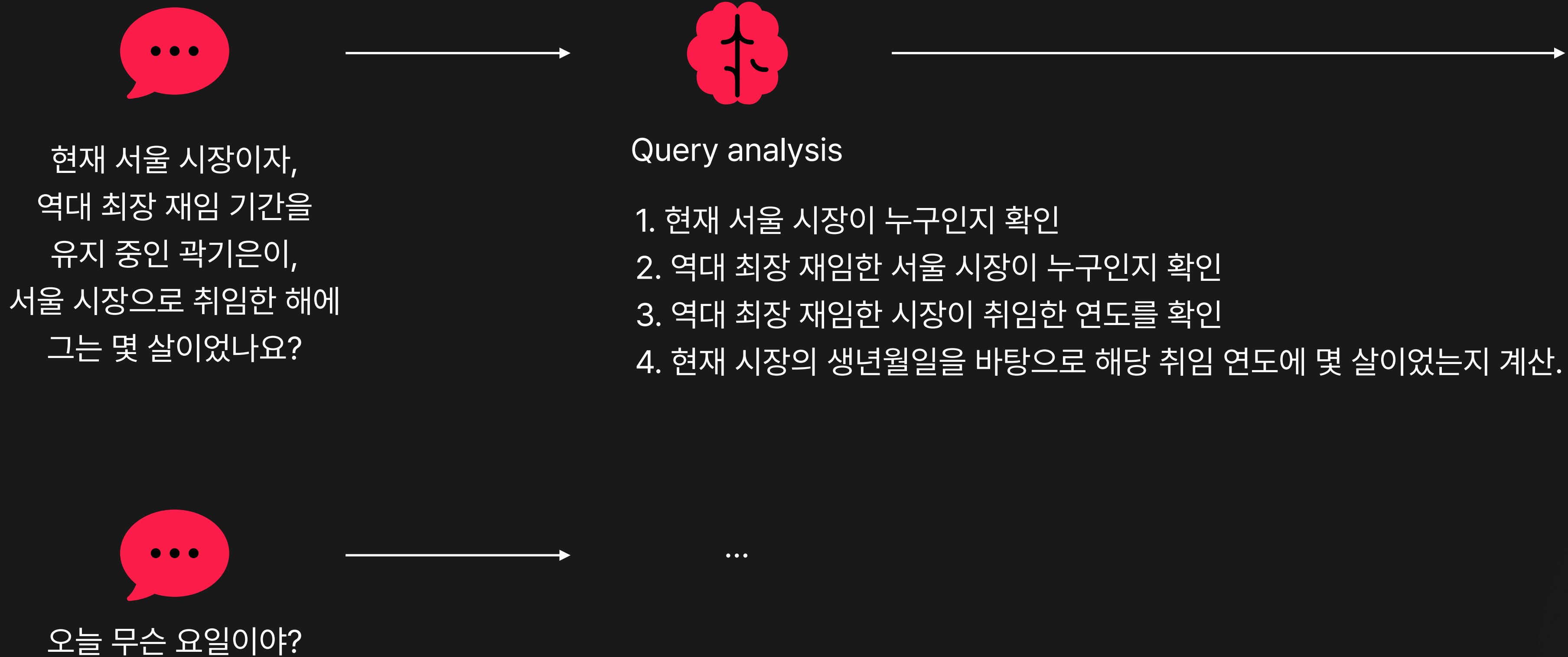
Tree Rag - reference

Arxiv 논문

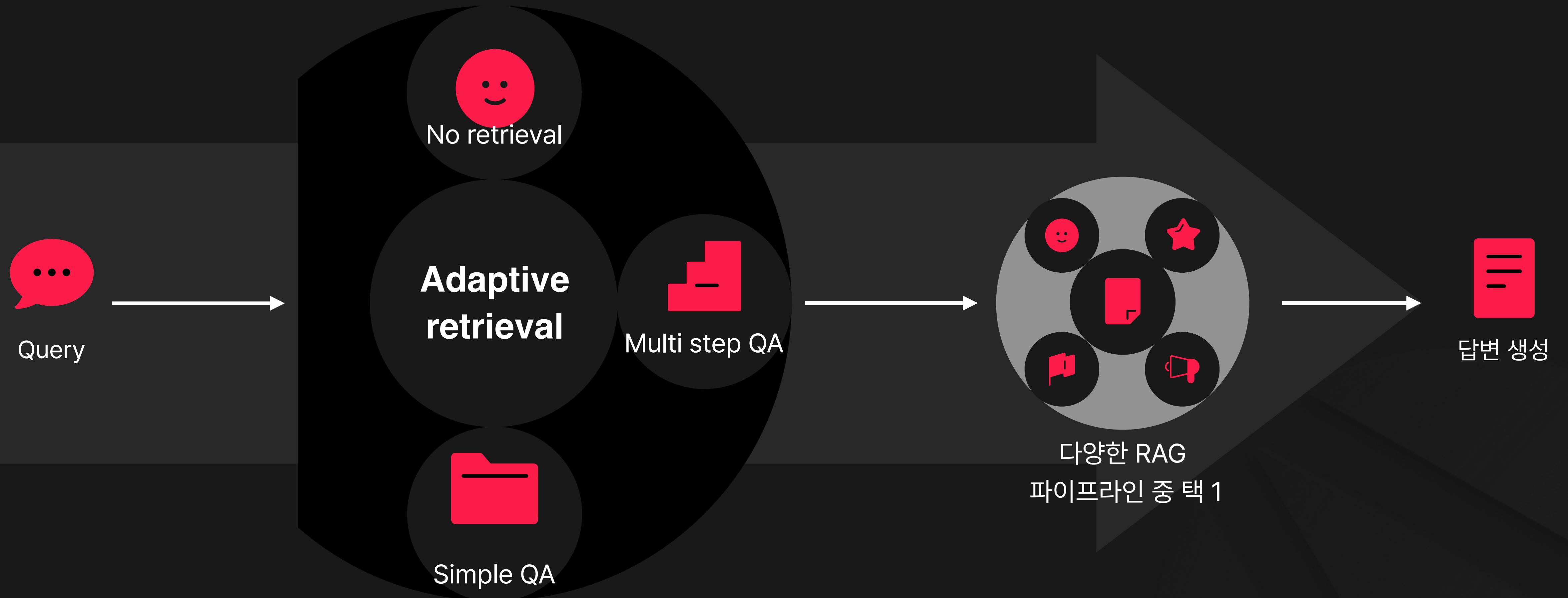
- <https://arxiv.org/pdf/2402.07483>

Adaptive RAG

Adaptive Rag - overview



Adaptive Rag - method

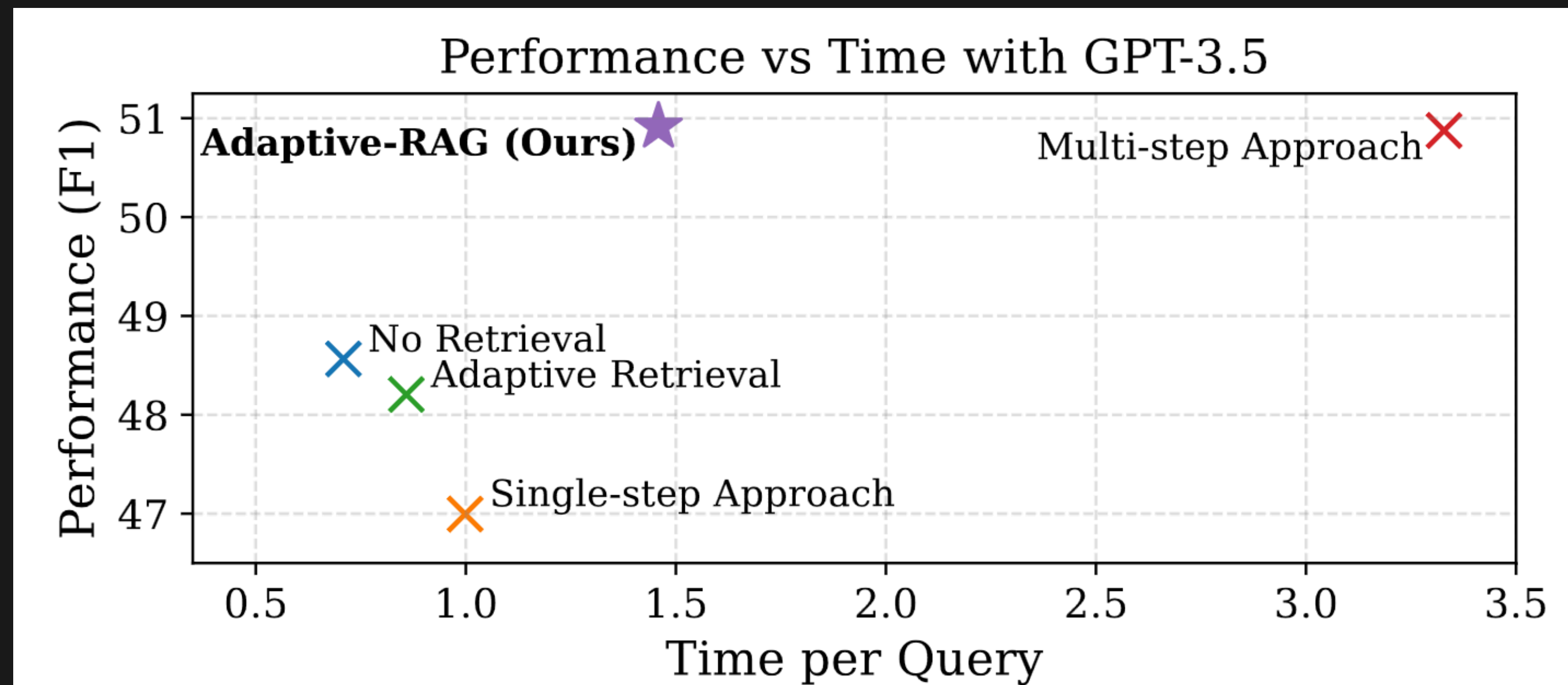


강점

효율성과 정확성을 모두 달성

<https://arxiv.org/pdf/2403.14403>

Adaptive Rag - results and performance

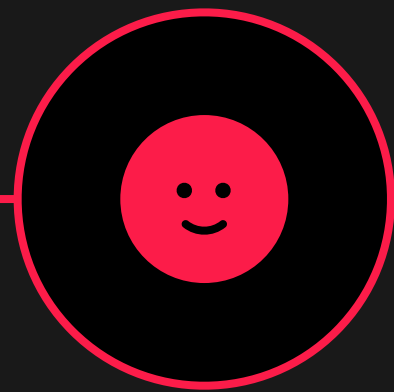


<https://arxiv.org/pdf/2403.14403>

Adaptive Rag - results and performance

Types	Methods	FLAN-T5-XL (3B)					FLAN-T5-XXL (11B)					GPT-3.5 (Turbo)				
		EM	F1	Acc	Step	Time	EM	F1	Acc	Step	Time	EM	F1	Acc	Step	Time
Simple	No Retrieval	14.87	21.12	15.97	0.00	0.11	17.83	25.14	19.33	0.00	0.08	35.77	48.56	44.27	0.00	0.71
	Single-step Approach	34.83	44.31	38.87	1.00	1.00	37.87	47.63	41.90	1.00	1.00	34.73	46.99	45.27	1.00	1.00
Adaptive	Adaptive Retrieval	23.87	32.24	26.73	0.50	0.56	26.93	35.67	29.73	0.50	0.54	35.90	48.20	45.30	0.50	0.86
	Self-RAG*	9.90	20.79	31.57	0.72	0.43	10.87	22.98	34.13	0.74	0.23	10.87	22.98	34.13	0.74	1.50
	Adaptive-RAG (Ours)	37.17	46.94	42.10	2.17	3.60	38.90	48.62	43.77	1.35	2.00	37.97	50.91	48.97	1.03	1.46
Complex	Multi-step Approach	39.00	48.85	43.70	4.69	8.81	40.13	50.09	45.20	2.13	3.80	38.13	50.87	49.70	2.81	3.33
Oracle	Adaptive-RAG w/ Oracle	45.00	56.28	49.90	1.28	2.11	47.17	58.60	52.20	0.84	1.10	47.70	62.80	58.57	0.50	1.03

Adaptive Rag - classifier



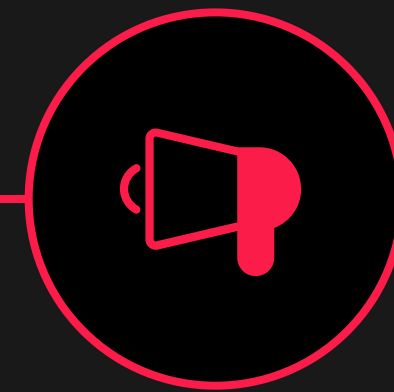
모델

Small Language Model



활용 데이터

- 레이블 A :
 - 가장 단순한 non-retrieval 방법으로 정답을 맞춤
- 레이블 B :
 - Single retrieval 방법으로 정답을 맞춤
 - Single hop retrieval
- 레이블 C :
 - Multi hop retrieval

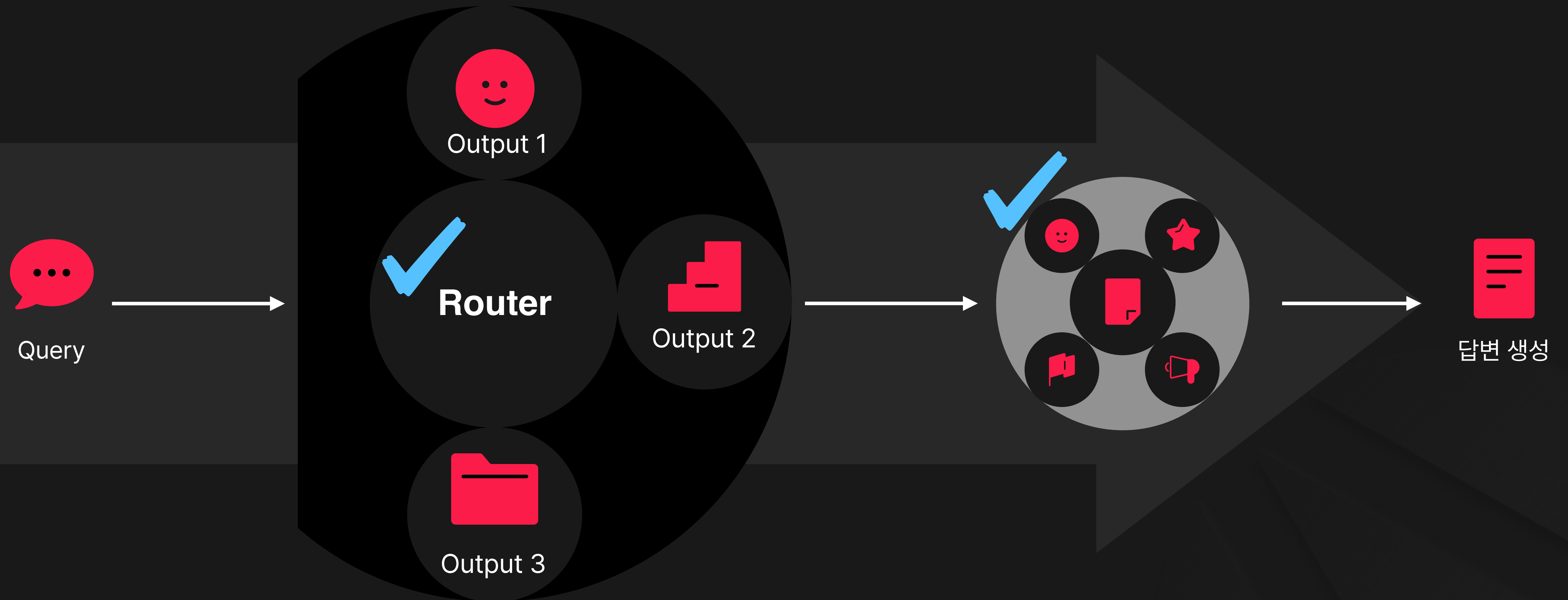


Training loss

cross-entropy loss

<https://arxiv.org/pdf/2403.14403>

Adaptive Rag - 응용 및 한계점



Adaptive Rag - reference

Langchain - code snippet

- https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_adaptive_rag.ipynb

Arxiv 논문

- <https://arxiv.org/pdf/2403.14403>

<https://arxiv.org/pdf/2403.14403>

Multi-hop RAG

Multi-hop Rag - overview

Why

다양한 출처의 정보를 활용한
질의응답 성능 평가

What

다양한 출처와 논리적인 사고를 통해
답변할 수 있는 QA set



Multi hop RAG

How

뉴스 api를 활용하여 데이터 수집
GPT를 활용, 논리적인 사고가 필요한 QA set 생성

활용 방안

Retrieval과 Generation 단계를
개별적으로 테스트

Multi-hop Rag - data stats

Category	Avg. Tokens	Entry Count
technology	2262.3	172
entertainment	2084.3	114
sports	2030.6	211
science	1745.5	21
business	1723.8	81
health	1481.1	10
total	2046.5	609

수집된 문서들의 주제

News api 활용

1024 토큰 이상의 아티클 수집

Multi-hop Rag - data stats

Query Category	Entry Count	Percentage
Inference Query	816	31.92%
Comparison Query	856	33.49%
Temporal Query	583	22.81%
Null Query	301	11.78%
Total	2,556	100.00 %

생성된 query type의 비율

추론이 필요한 쿼리 : (단답형)

- 2019, 2020 애플의 실적 보고서 중 공급망 이슈가 제기된 보고서는?

비교가 필요한 쿼리 : (단답형)

- 넷플릭스와 구글 중 2023년에 더 높은 매출을 올린 회사는?

시간 및 전후 과정을 평가하는 쿼리 : (yes/no/after/before)

- 애플이 airtag 기능을 iPad pro 5세대 런칭 전에 발표했는가?

Null 쿼리 :

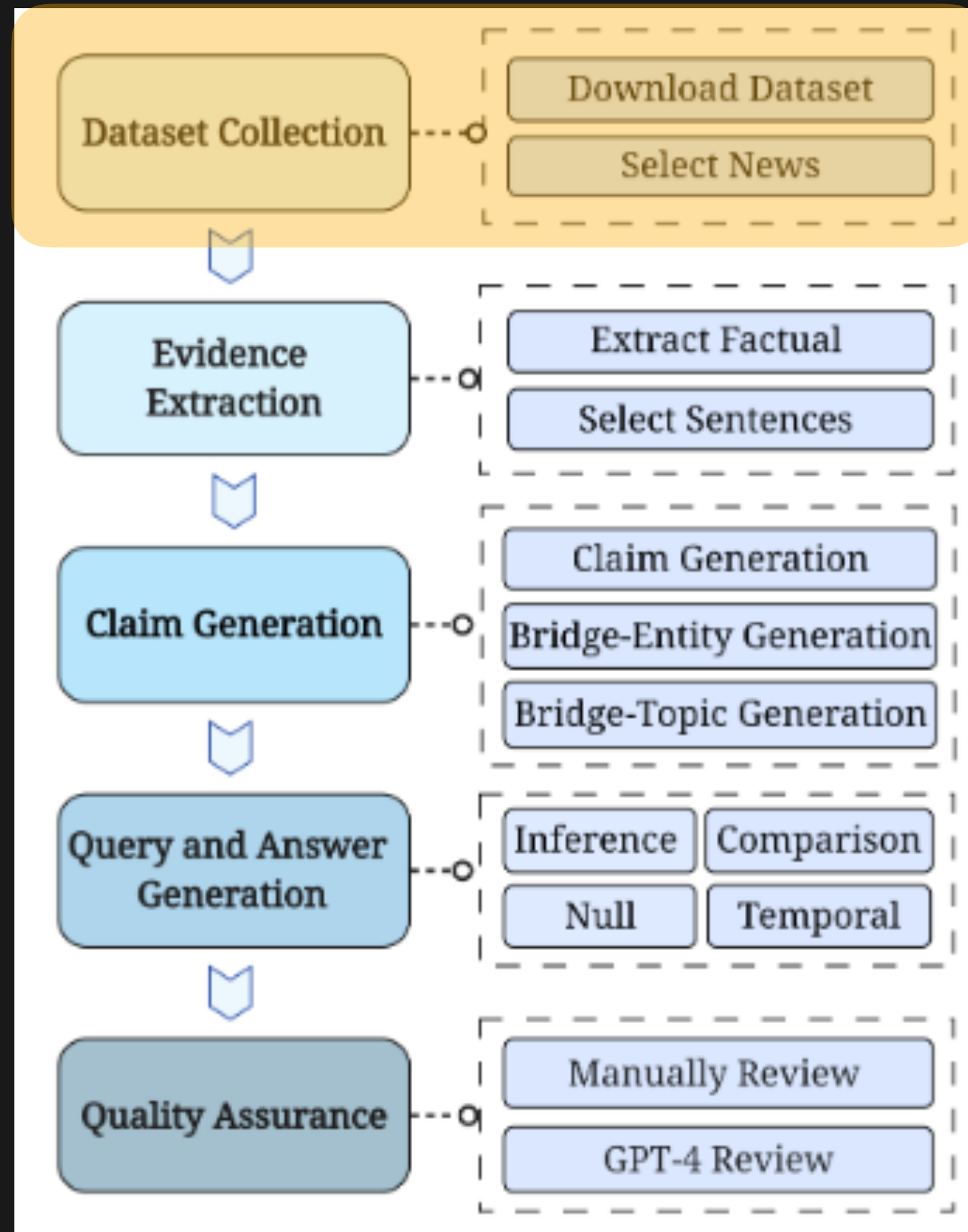
- 현재 문서로 답변을 할 수 없는 쿼리 (Hallucination 평가)

Multi-hop Rag - data stats

Num. of Evidence Needed	Count	Percentage
0 (Null Query)	301	11.78%
2	1078	42.18%
3	779	30.48%
4	398	15.56%
Total	2,556	100.00 %

정답 생성을 위해 필요한 chunk 개수

Multi-hop Rag - 데이터셋 구축 과정



01

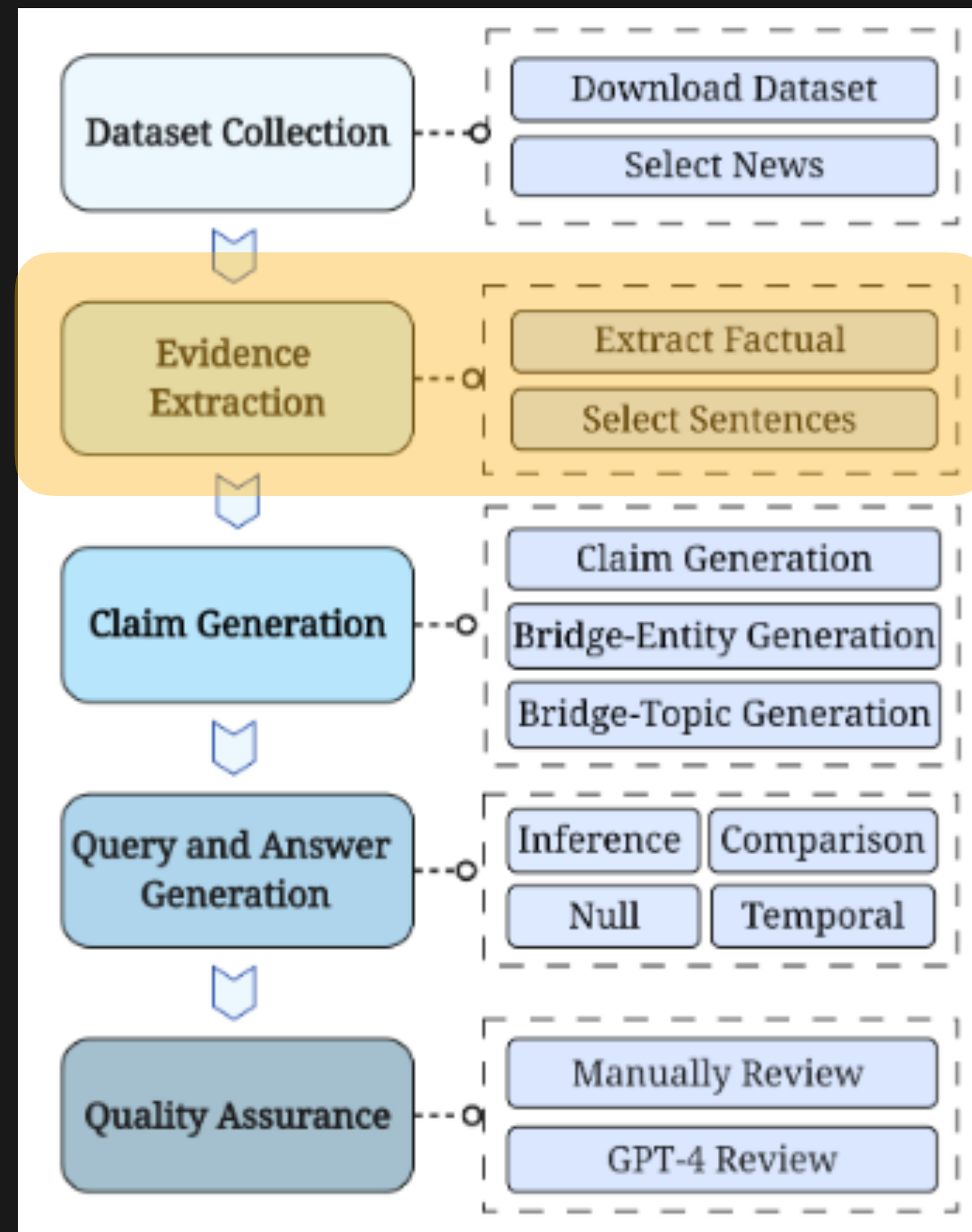
News api

Mediastack API 활용

LLM이 학습하지 못한 2023.09~2023.12 뉴스 (token length $\geq 1,024$)

날짜, 저자, 카테고리, URL 등은 메타데이터에 저장

Multi-hop Rag - 데이터셋 구축 과정

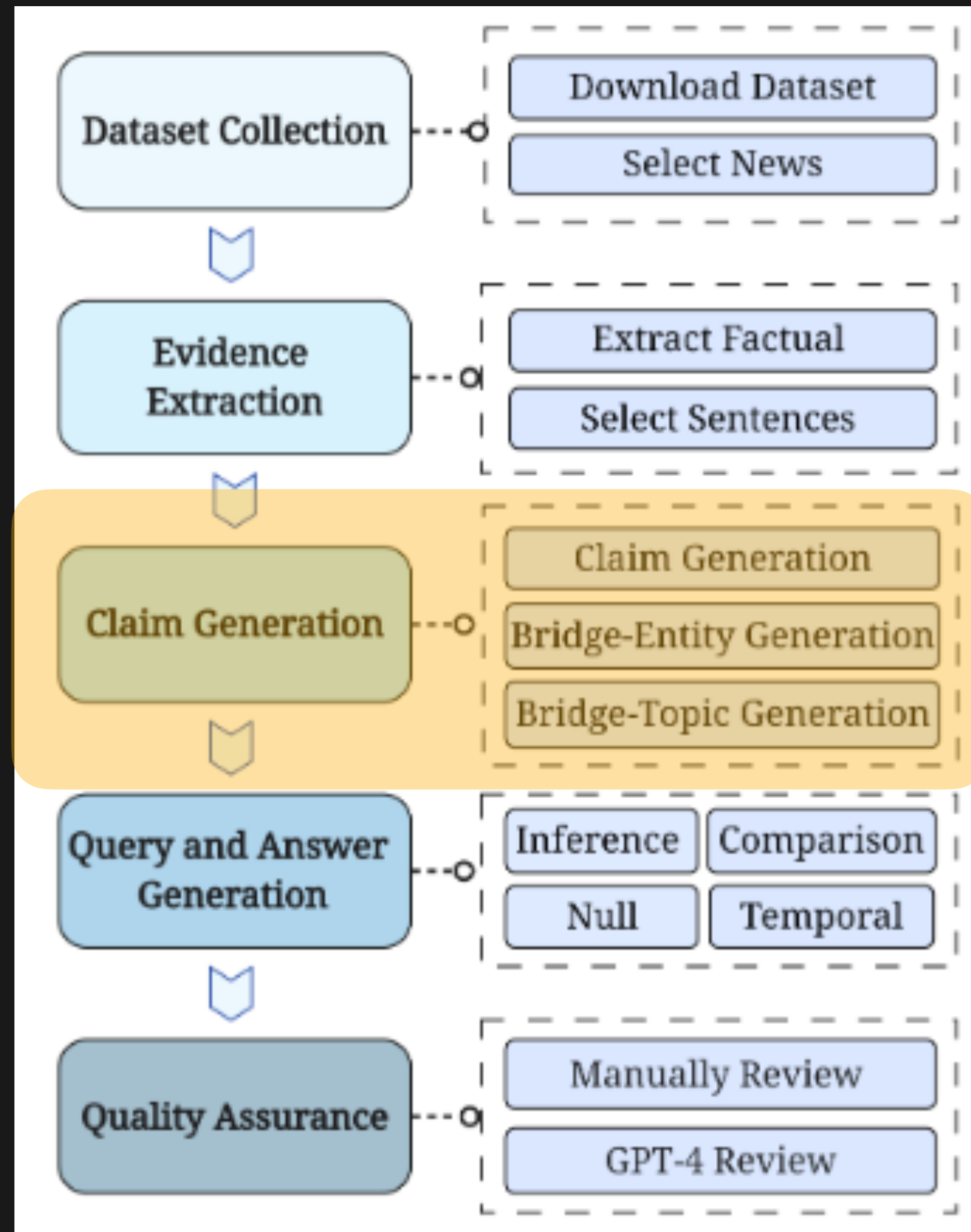


02

Evidence 추출

뉴스 내에서 문장 추출

Multi-hop Rag - 데이터셋 구축 과정



03

Claim generation

(문장 추출)

→ Back then, just like today, home prices had boomed for years before Fed officials were ultimately forced to hike interest rates aggressively in an attempt to fight inflation.

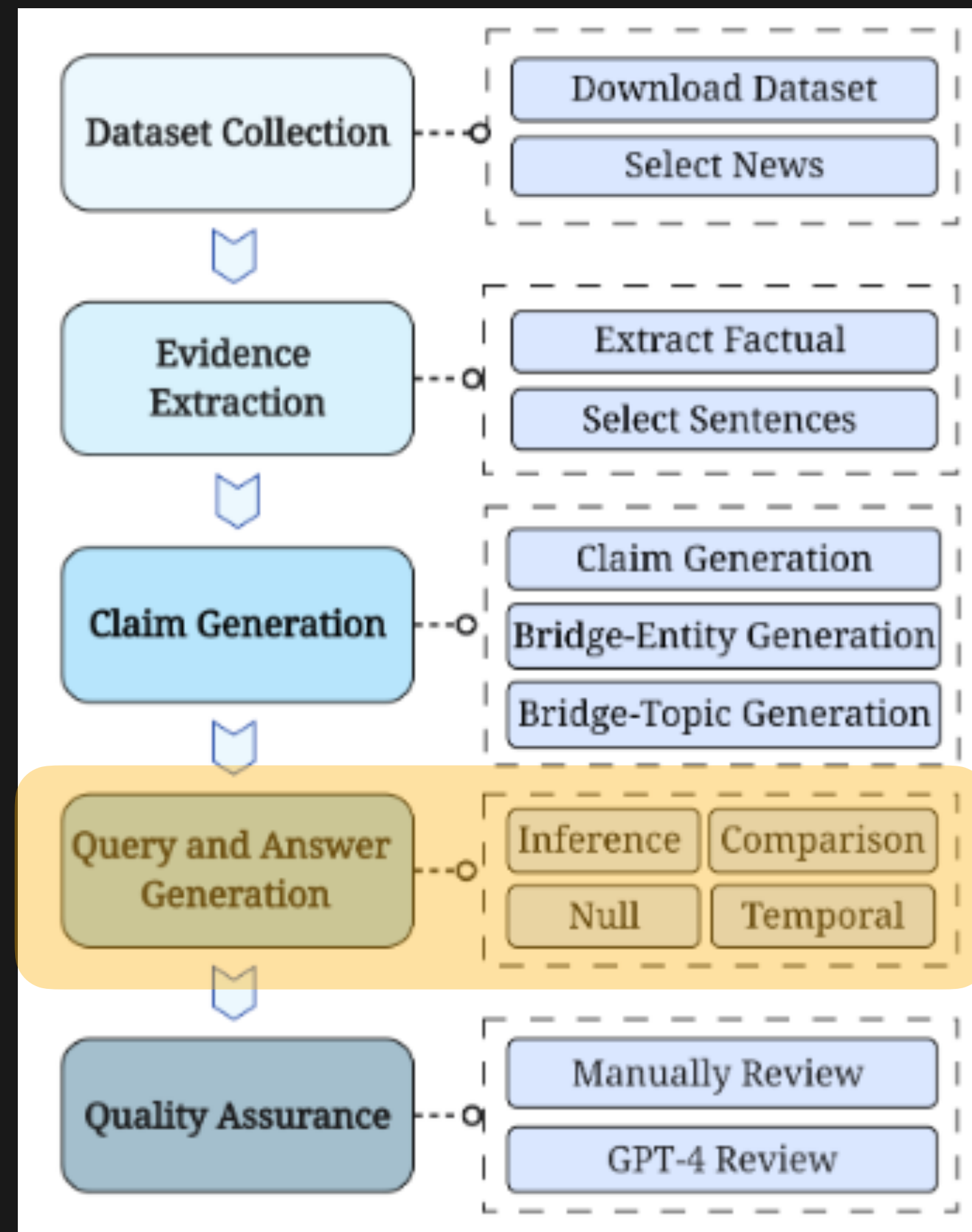
(문장 재작성)

→ Federal Reserve officials were forced to aggressively hike interest rates to combat inflation after years of booming home prices

(Bridge entity & topic 생성)

→ "Federal Reserve", "Interest rate hikes to combat inflation"

Multi-hop Rag - 데이터셋 구축 과정

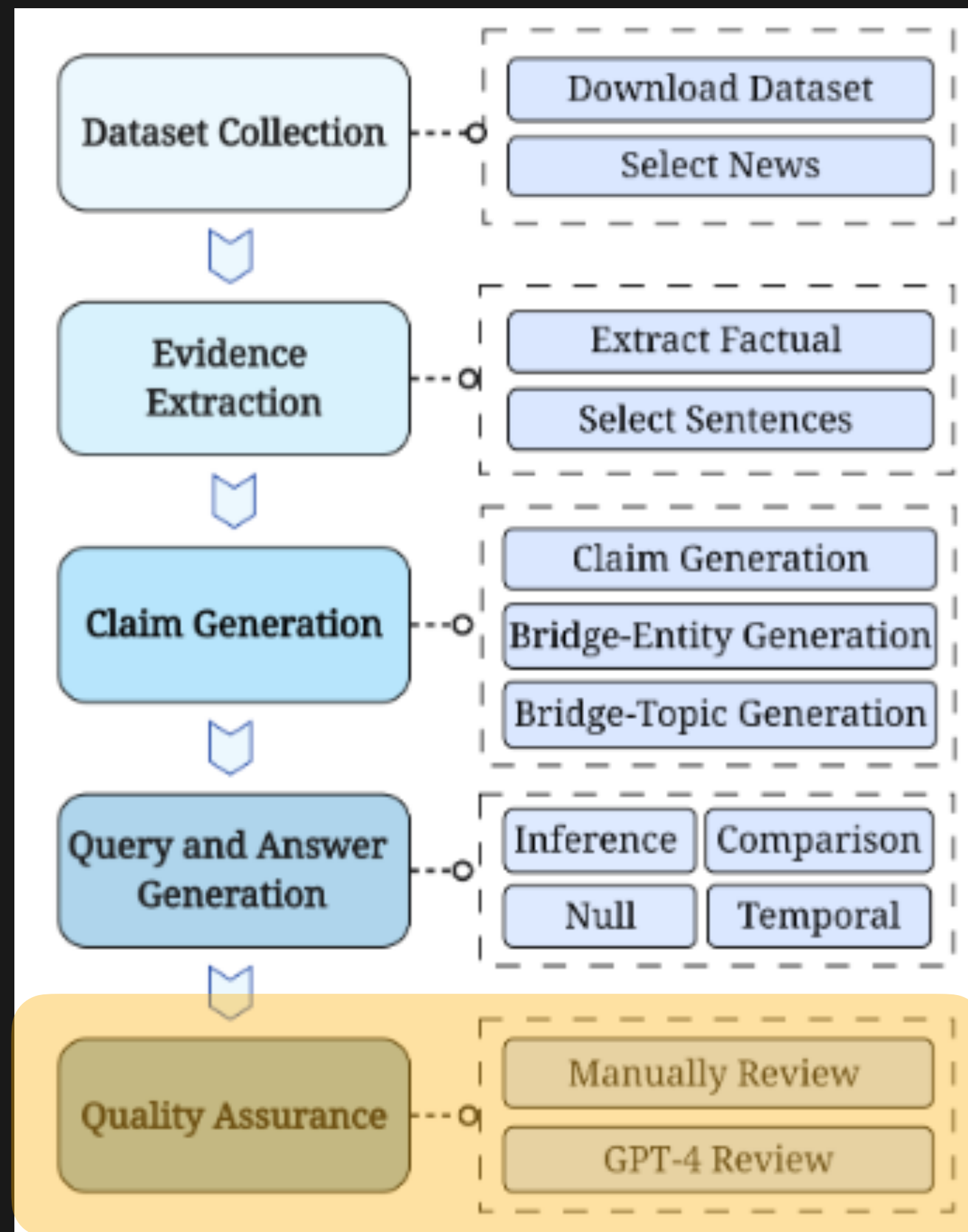


04

Query generation

- 생성된 bridge-topic과 bridge entity를 공유하는 claim들을 그룹화
- 4가지 유형의 쿼리 생성
- GPT-4

Multi-hop Rag - 데이터셋 구축 과정



04

생성된 답변 검증

1. 생성된 multi-hop query, 근거 문서 등을 사람이 직접 검수
2. GPT-4에 다음 기준을 제공하여 검수
 - 쿼리는 제공된 모든 근거를 사용해야 한다
 - 쿼리는 근거만을 활용해 답변을 해야 한다
 - 쿼리는 한 단어 또는 특정 개체로 대답해야 한다
 - 쿼리는 지정된 질문 유형과 부합해야 한다

Multi-hop Rag - 예시

News source	Fortune Magazine	The Sydney Morning Herald
Evidence	Back then, just like today, home prices had boomed for years before Fed officials were ultimately forced to hike interest rates aggressively in an attempt to fight inflation.	Postponements of such reports could complicate things for the Fed, which has insisted it will make upcoming decisions on interest rates based on what incoming data say about the economy.
Claim	Federal Reserve officials were forced to aggressively hike interest rates to combat inflation after years of booming home prices.	The Federal Reserve has insisted that it will base its upcoming decisions on interest rates on the incoming economic data.
Bridge-Topic	Interest rate hikes to combat inflation	Interest rate decisions based on economic data
Bridge-Entity	Federal Reserve	Federal Reserve
Query	Does the article from Fortune suggest that the Federal Reserve's interest rate hikes are a response to past conditions, such as booming home prices, while The Sydney Morning Herald article indicates that the Federal Reserve's future interest rate decisions will be based on incoming economic data?	
Answer	Yes	

Multi-hop Rag - reference

Git

- <https://github.com/yixuantt/MultiHop-RAG/>

Arxiv 논문

- <https://arxiv.org/pdf/2401.15391>

<https://arxiv.org/pdf/2401.15391>

Modular rag components

Modular rag - indexing

Chunk optimization

Small to big

Sliding window

Semantic chunking

Metadata

Index optimization

Hierarchical index

Knowledge graph

Modular rag - 검색 전

쿼리 변환

Rewrite

Step back prompting

HyDE

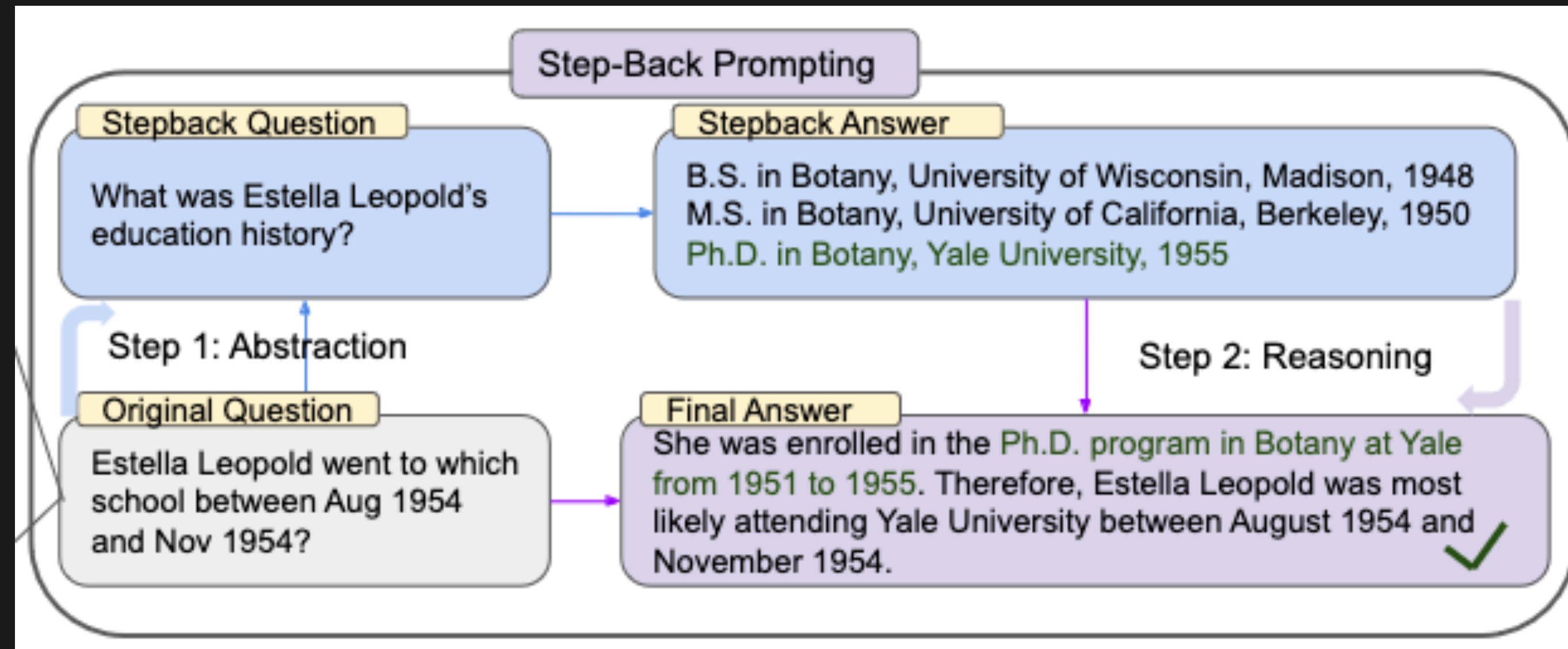
쿼리 확장

Multi query

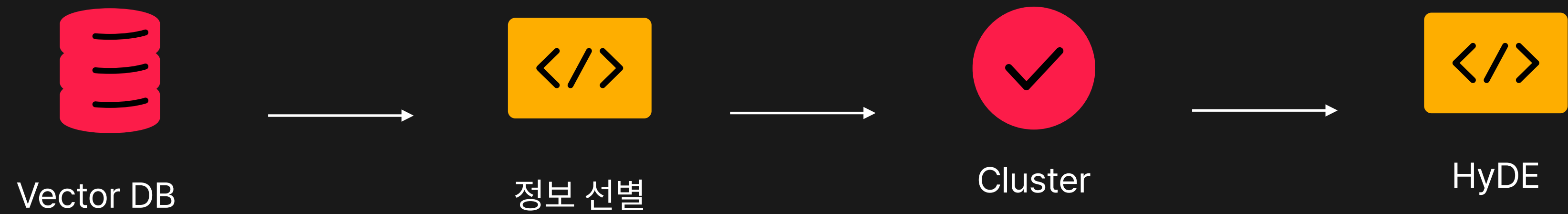
Sub query

CoVE

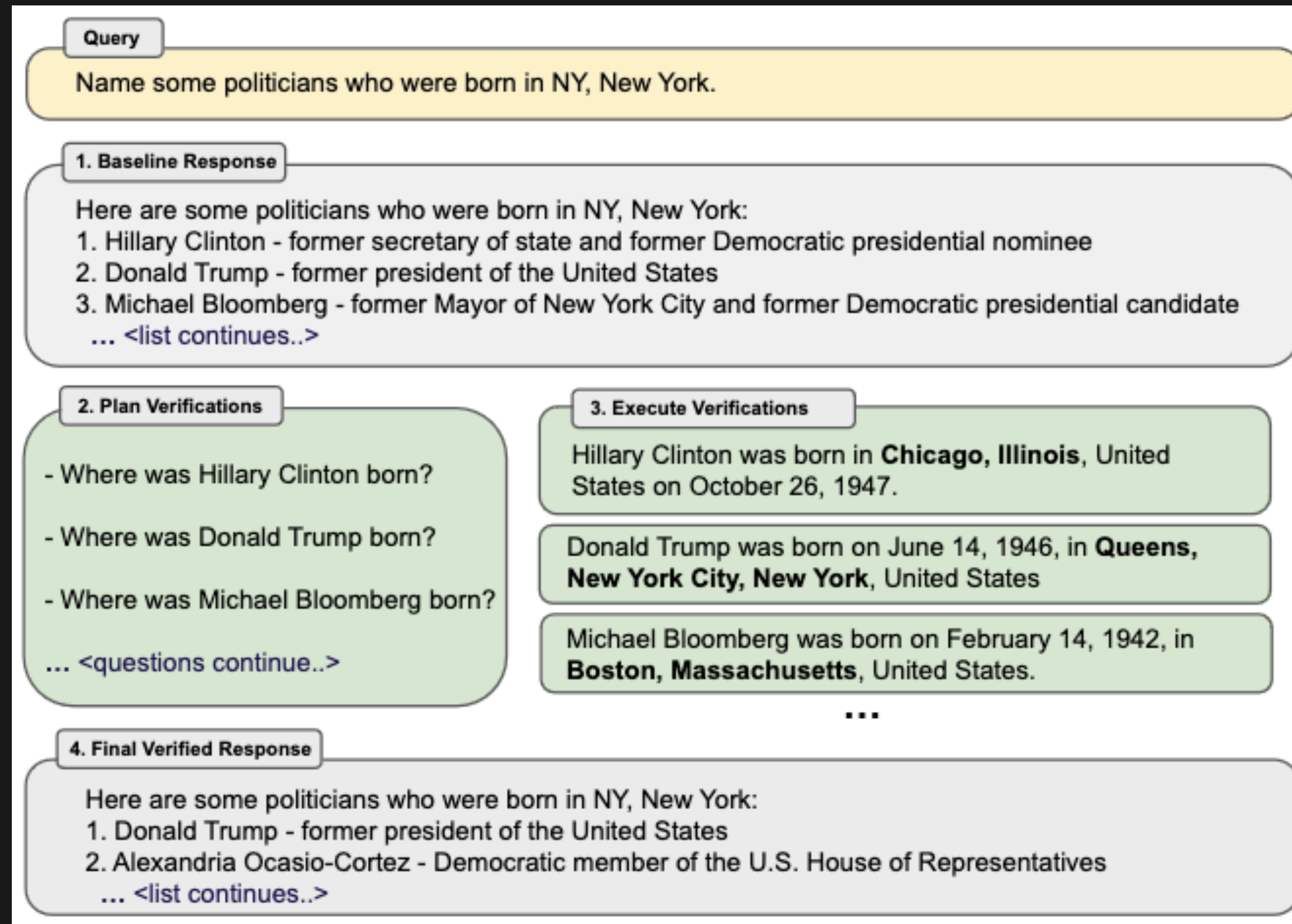
Modular rag - 검색 전 (Step back prompting)



Modular rag - 검색 전 (HyDE -> AutoHyDE)



Modular rag - 검색 전 (CoVE)



Modular rag - 검색

검색 알고리즘

Sparse retriever

Dense retriever

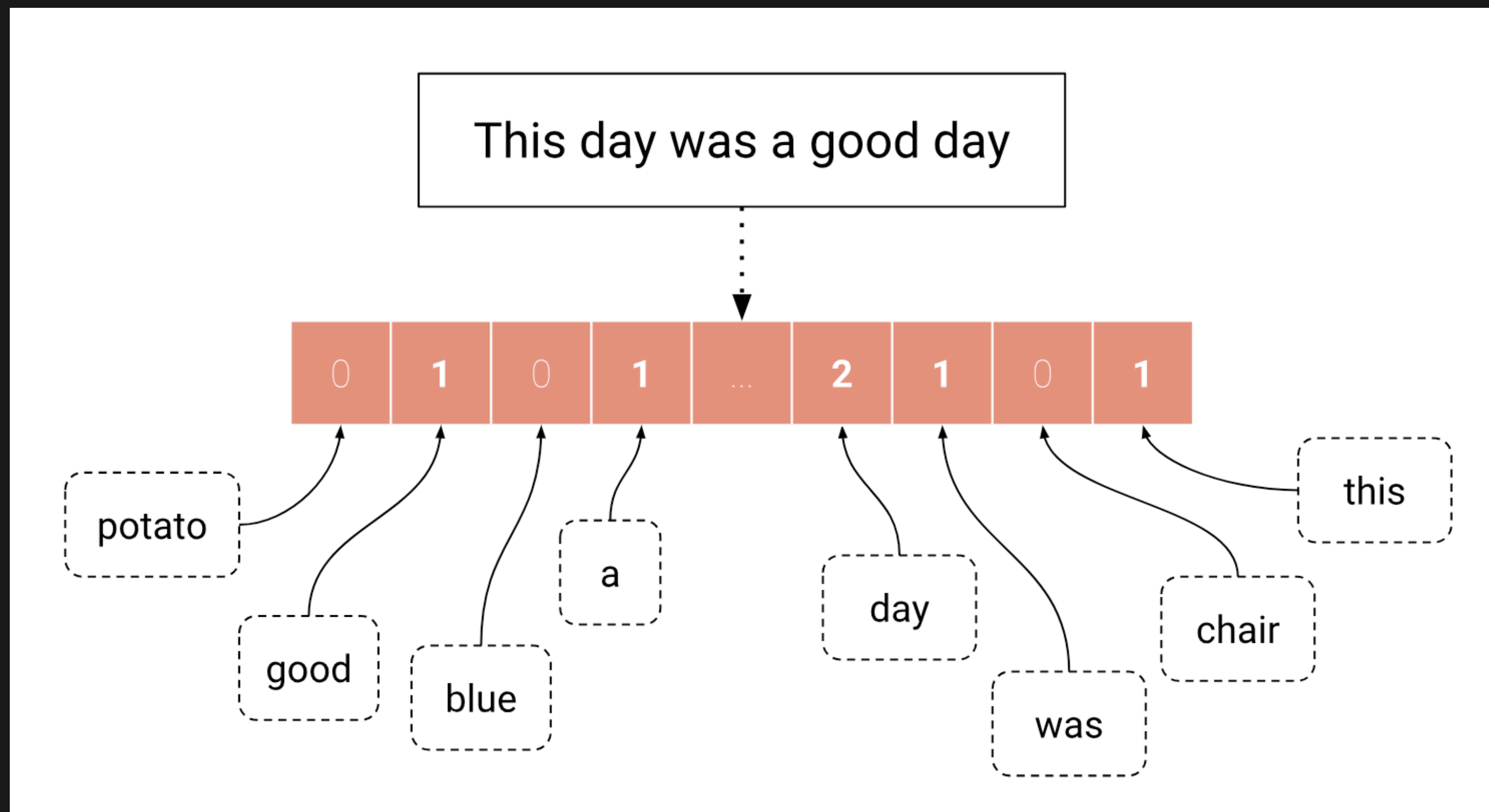
Mixed retriever

검색기 튜닝

SFT

Adapter

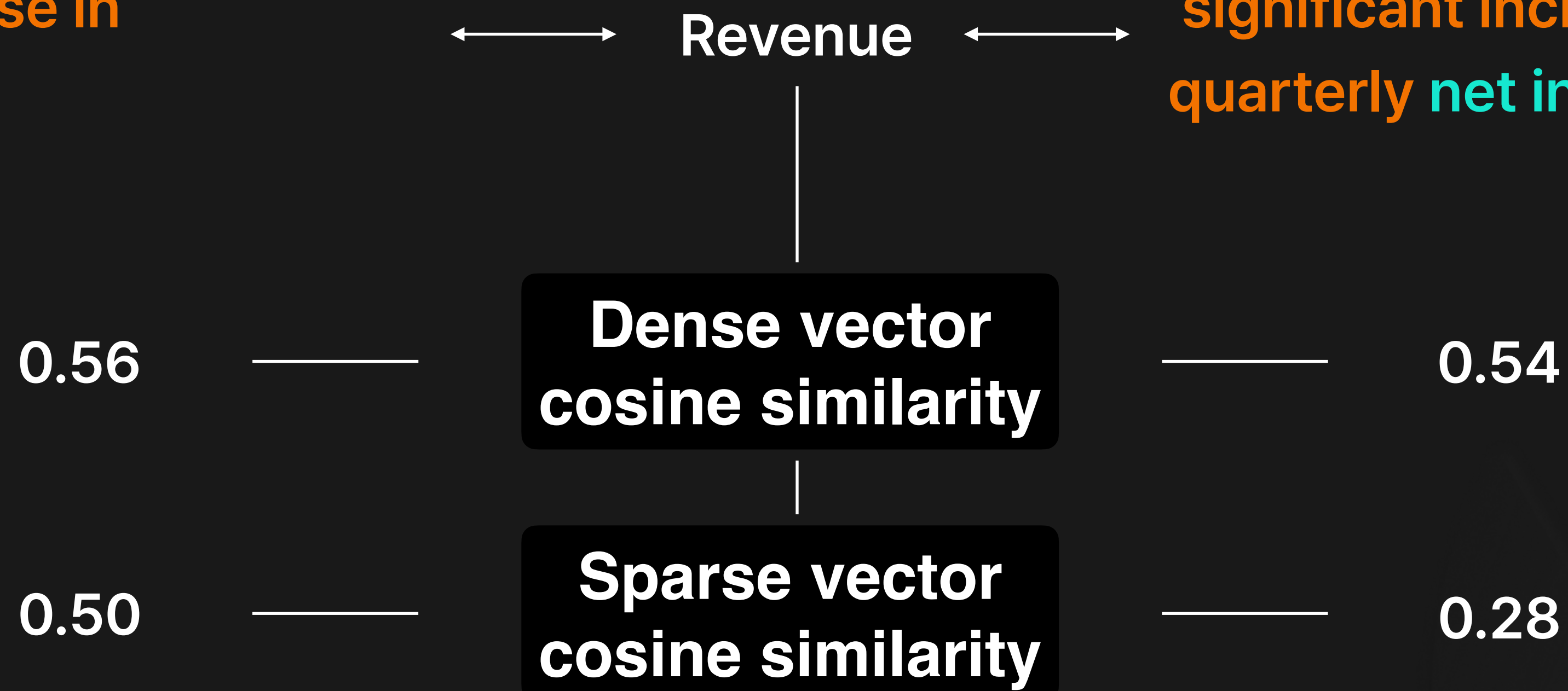
Sparse vector란?



Sparse vector를 사용하는 이유

The financial report highlights a significant increase in quarterly revenue

The financial report highlights a significant increase in quarterly net income



Sparse vector를 활용한 서치 - TF-IDF & BM25

Term frequency

- 단어 출현 횟수 / 문서의 단어 개수
- 하지만,
긴 문서 = 단어들의 빈도 증가
- 따라서 문서 길이 정규화 &
단어 빈도 정규화

Inverse doc frequency

- (전체 문서 개수) / (해당 단어가 들어있는 문서의 개수)
- 단어의 uniqueness 가중치

Document 1

Term	Term Count
this	1
is	1
a	2
sample	1

Document 2

Term	Term Count
this	1
is	1
another	2
example	3

$$\text{tf}(\text{"this"}, d_1) = \frac{1}{5} = 0.2$$

$$\text{tf}(\text{"this"}, d_2) = \frac{1}{7} \approx 0.14$$

$$= \log\left(\frac{N}{\text{The number of documents containing that term}}\right)$$

$$\text{idf}(\text{"this"}, D) = \log\left(\frac{2}{2}\right) = 0$$

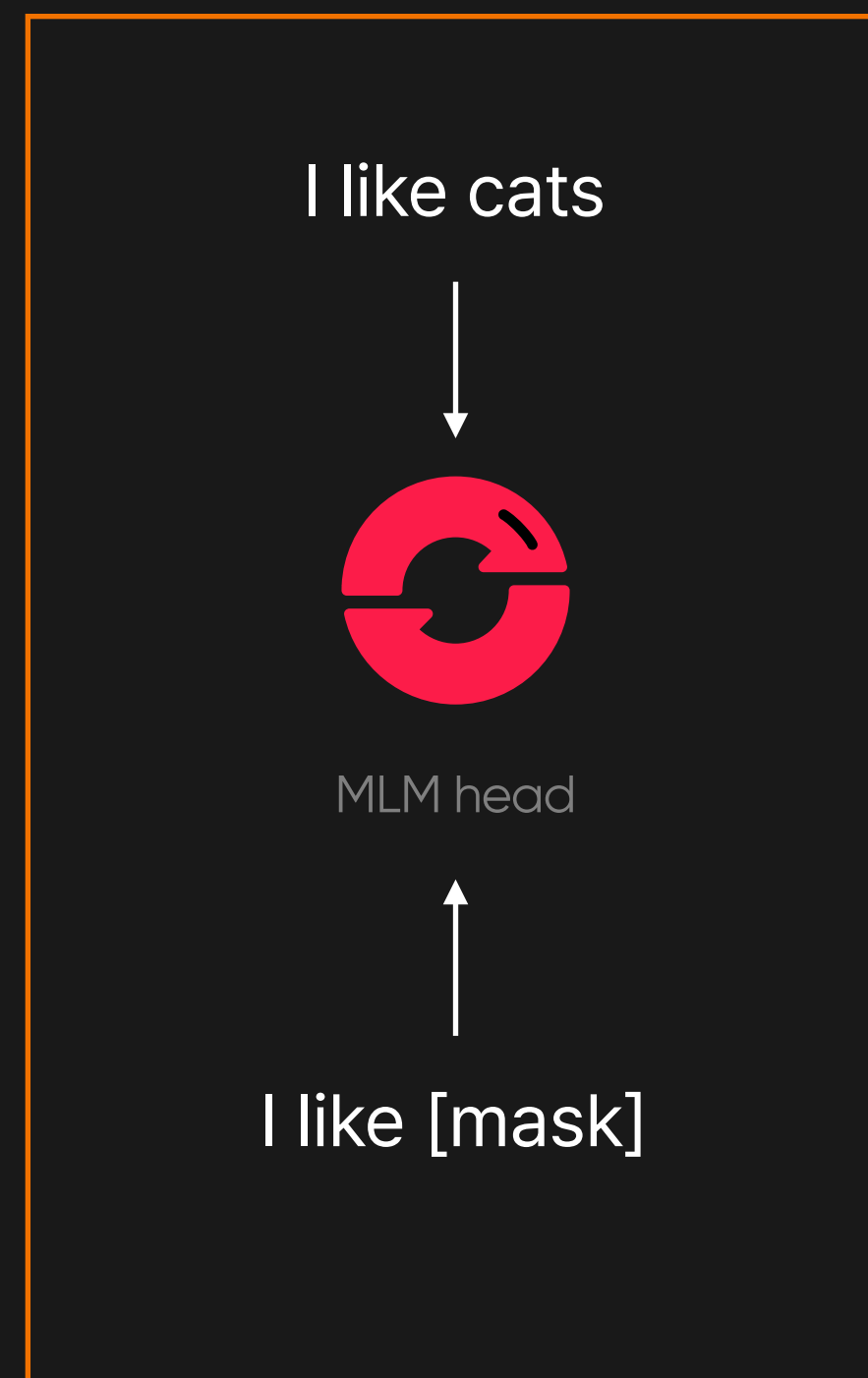
Sparse vector를 활용한 서치의 한계점 - word mismatch

Term Expansion

- BERT의 MLM(Mask Language Modeling head)을 활용 문장에 랜덤 mask를 생성하여 feed.

MLM head에는 original un-masked sentence를 제공.

각 mask 별로 예측 proba 제공
→ term expansion



Modular rag - 검색 후

순위 재배열

Rule based

Model based

LLM based

RRF

압축

LLMLingua

Recomp

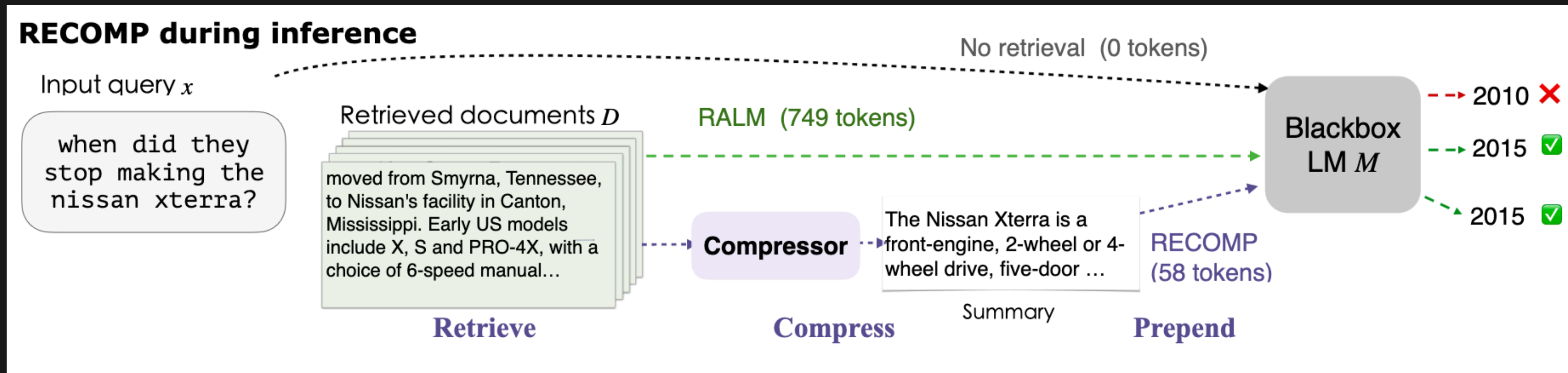
Selective context

LLM-Critique

Modular rag - 검색 후 (LLMLingua)



Modular rag - 검색 후 (Recomp)



01

문서 검색

02

압축

1. Extractive
2. Abstractive

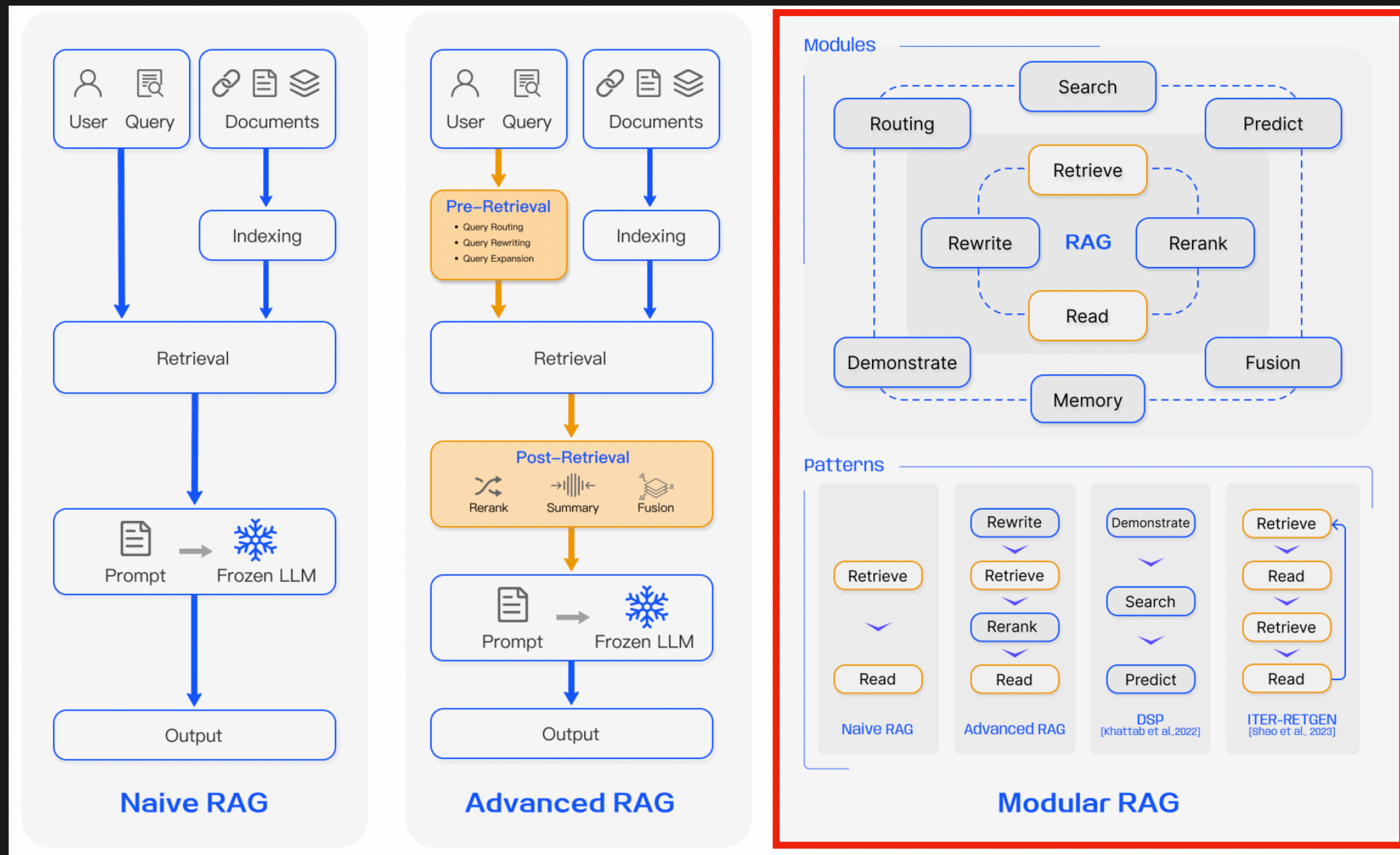
03

선별적
augmentation

04

Generate

Modular rag



<https://arxiv.org/pdf/2312.10997>